

# Lecture 13:

## Generative Models (part 2)

# Last Time: Generative vs Discriminative Models

## **Discriminative Model:**

Learn a probability distribution  $p(y|x)$

## **Generative Model:**

Learn a probability distribution  $p(x)$

## **Conditional Generative Model:**

Learn  $p(x|y)$

**Data:**  $x$



**Label:**  $y$

Cat

## **Density Function**

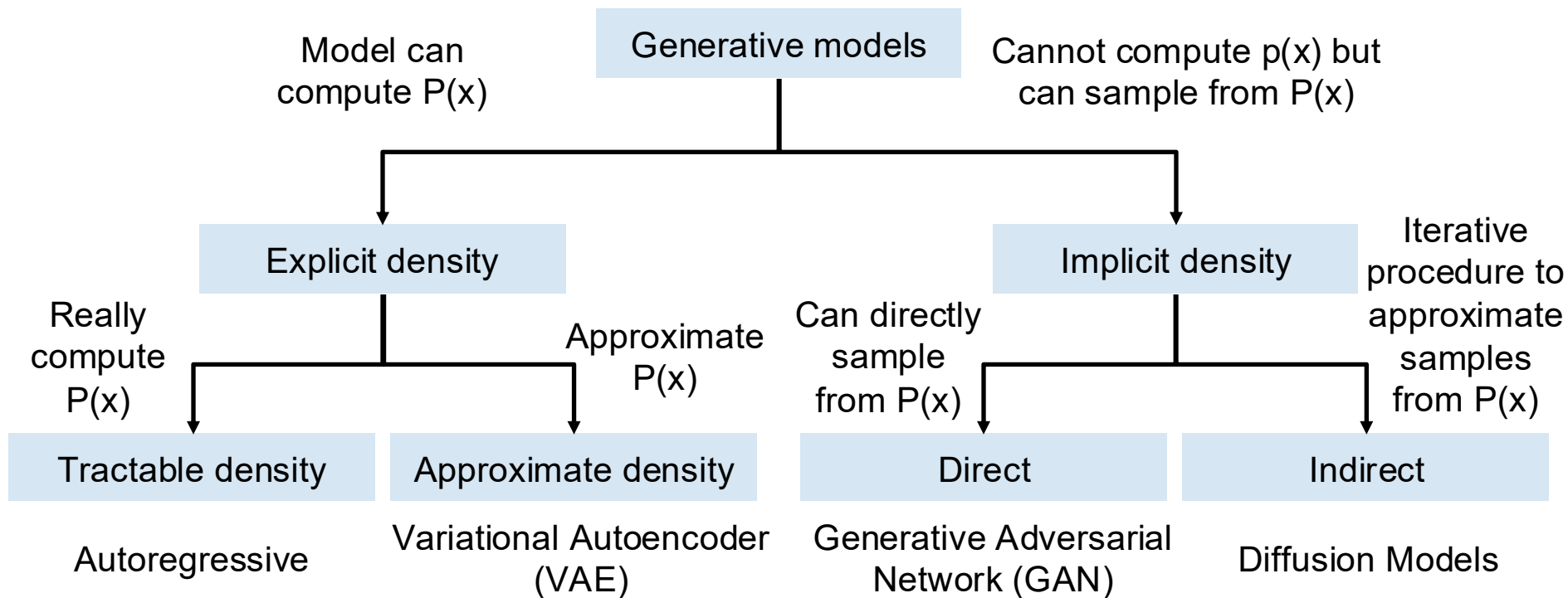
$p(x)$  assigns a positive number to each possible  $x$ ; higher numbers mean  $x$  is more likely.

Density functions are **normalized**:

$$\int_x p(x) dx = 1$$

Different values of  $x$  **compete** for density

# Last Time: Generative Models

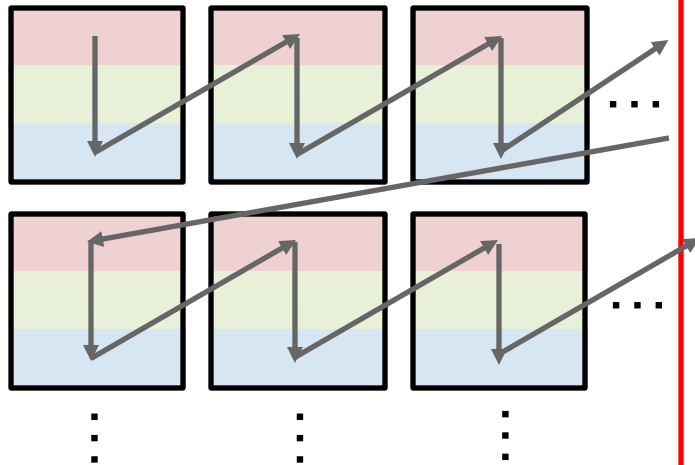
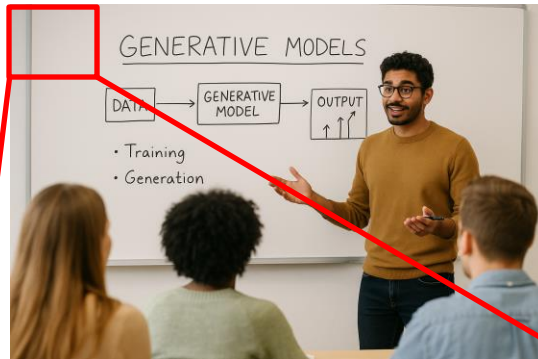


# Last Time: Autoregressive Models

Treat data as a sequence  
(e.g. image as sequence of pixels)

$$\begin{aligned} p(x) &= p(x_1, x_2, \dots, x_N) \\ &= p(x_1)p(x_2 | x_1)p(x_3 | x_1, x_2) \dots \\ &= \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1}) \end{aligned}$$

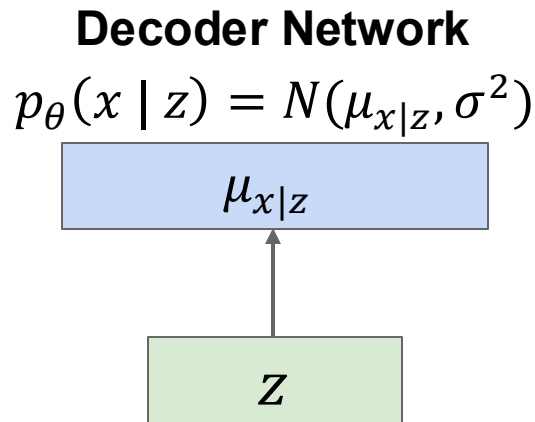
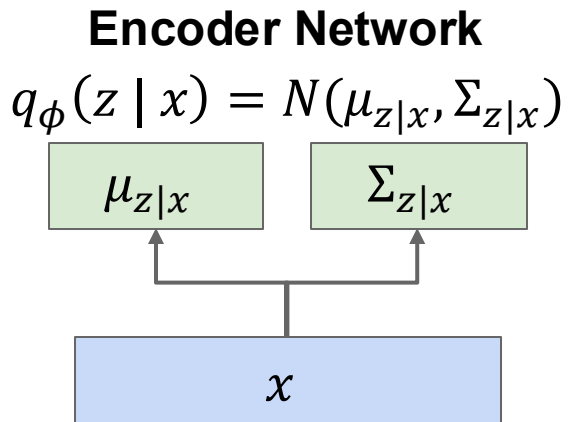
Model with an RNN or Transformer



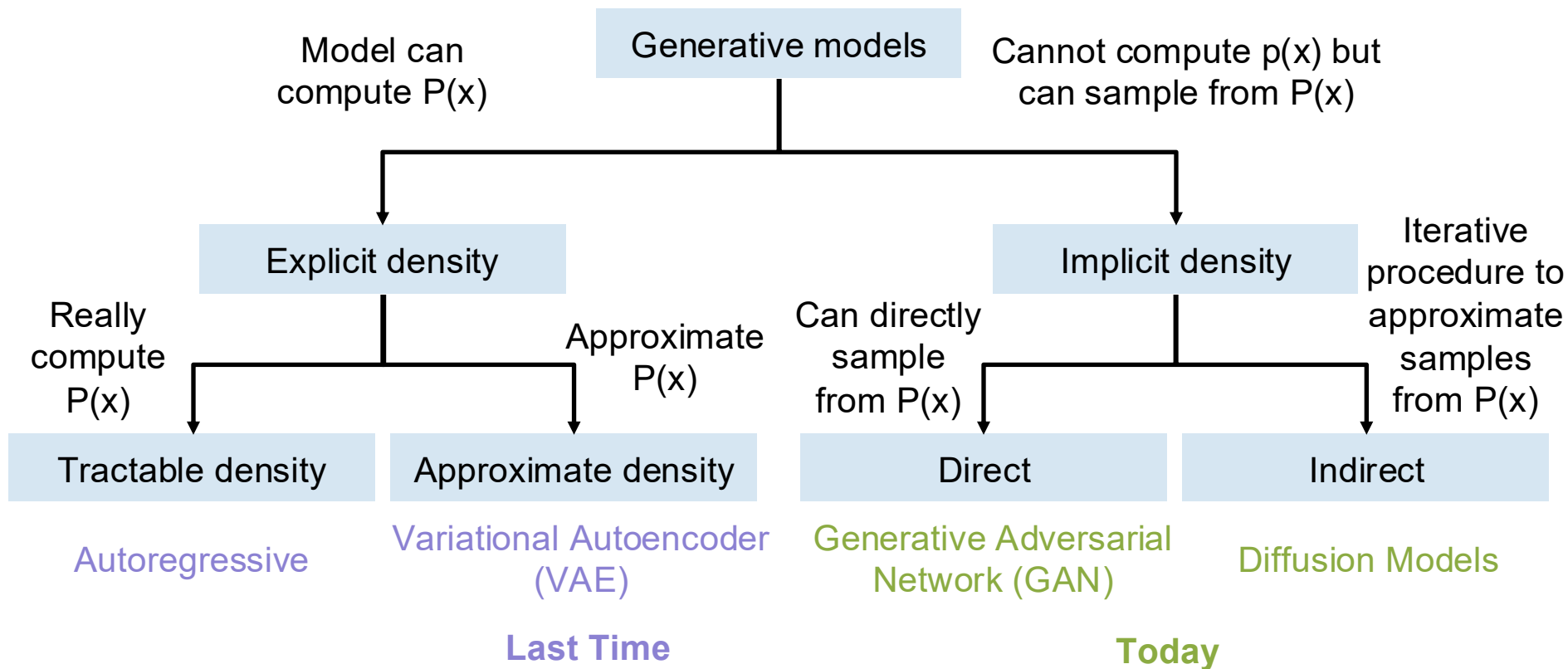
# Last Time: Variational Autoencoders

Jointly train **encoder**  $q$  and **decoder**  $p$  to maximize the **variational lower bound** on the data likelihood  
Also called **Evidence Lower Bound (ELBo)**

$$\log p_{\theta}(x) \geq E_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - D_{KL} \left( q_{\phi}(z|x), p(z) \right)$$



# Today: More Generative Models



# Generative Adversarial Networks (GANs)

# Generative Models So Far

**Autoregressive Models** directly maximize likelihood of training data:

$$p_{\theta}(x) = \prod_{i=1}^N p_{\theta}(x_i | x_1, \dots, x_{i-1})$$

**Variational Autoencoders** introduce a latent  $z$ , and maximize a lower bound:

$$p_{\theta}(x) = \int_Z p_{\theta}(x|z)p(z)dz \geq E_{z \sim q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL}\left(q_{\phi}(z|x), p(z)\right)$$

# Generative Models So Far

**Autoregressive Models** directly maximize likelihood of training data:

$$p_{\theta}(x) = \prod_{i=1}^N p_{\theta}(x_i | x_1, \dots, x_{i-1})$$

**Variational Autoencoders** introduce a latent  $z$ , and maximize a lower bound:

$$p_{\theta}(x) = \int_Z p_{\theta}(x|z)p(z)dz \geq E_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x), p(z))$$

**Generative Adversarial Networks** give up on modeling  $p(x)$ , but allow us to draw samples from  $p(x)$

# Generative Adversarial Networks

**Setup:** Have data  $x_i$  drawn from distribution  $p_{\text{data}}(x)$ . Want to sample from  $p_{\text{data}}$

# Generative Adversarial Networks

**Setup:** Have data  $x_i$  drawn from distribution  $p_{\text{data}}(x)$ . Want to sample from  $p_{\text{data}}$

**Idea:** Introduce a latent variable  $z$  with simple prior  $p(z)$  (e.g. unit Gaussian)

Sample  $z \sim p(z)$  and pass to a **Generator Network**  $x = G(z)$

Then  $x$  is a sample from the **Generator distribution**  $p_G$ . Want  $p_G = p_{\text{data}}$ !

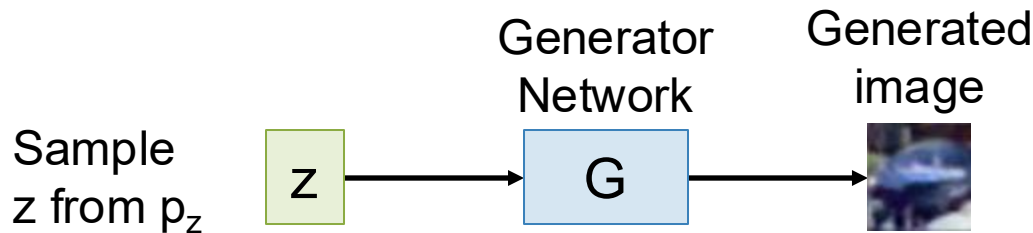
# Generative Adversarial Networks

**Setup:** Have data  $x_i$  drawn from distribution  $p_{\text{data}}(x)$ . Want to sample from  $p_{\text{data}}$

**Idea:** Introduce a latent variable  $z$  with simple prior  $p(z)$  (e.g. unit Gaussian)

Sample  $z \sim p(z)$  and pass to a **Generator Network**  $x = G(z)$

Then  $x$  is a sample from the **Generator distribution**  $p_G$ . Want  $p_G = p_{\text{data}}$ !



Train **Generator Network**  $G$  to convert  
 $z$  into fake data  $x$  sampled from  $p_G$

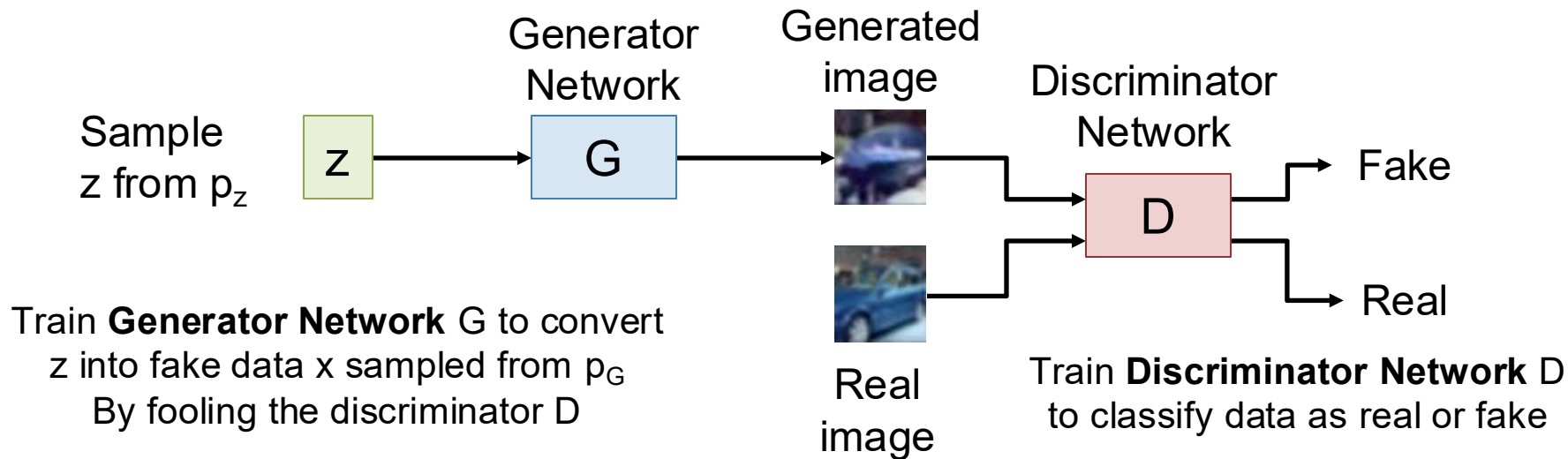
# Generative Adversarial Networks

**Setup:** Have data  $x_i$  drawn from distribution  $p_{\text{data}}(x)$ . Want to sample from  $p_{\text{data}}$

**Idea:** Introduce a latent variable  $z$  with simple prior  $p(z)$  (e.g. unit Gaussian)

Sample  $z \sim p(z)$  and pass to a **Generator Network**  $x = G(z)$

Then  $x$  is a sample from the **Generator distribution**  $p_G$ . Want  $p_G = p_{\text{data}}$ !



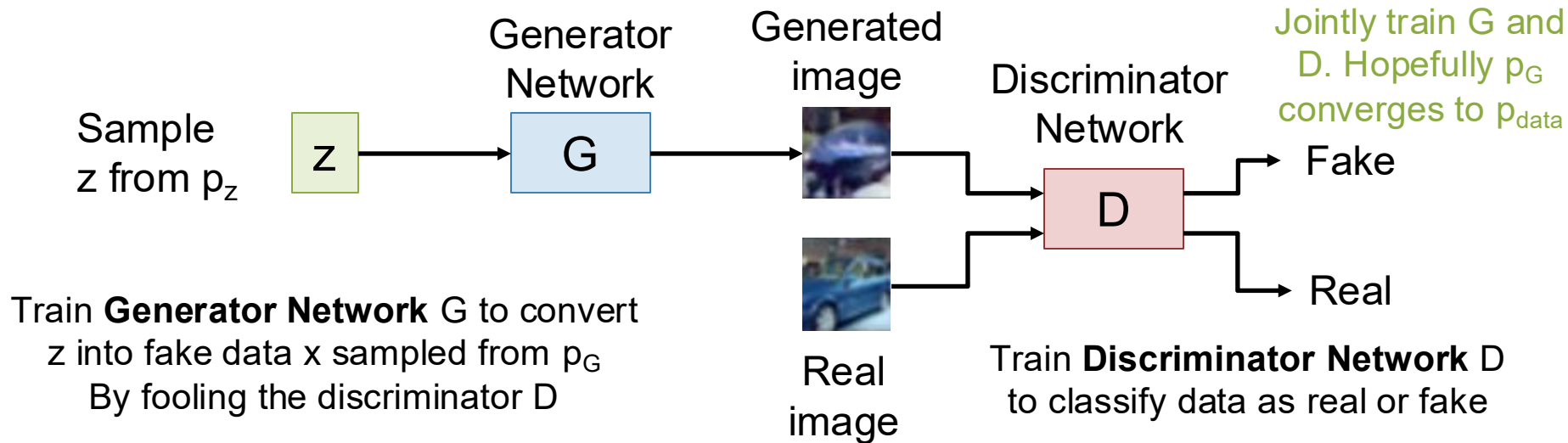
# Generative Adversarial Networks

**Setup:** Have data  $x_i$  drawn from distribution  $p_{\text{data}}(x)$ . Want to sample from  $p_{\text{data}}$

**Idea:** Introduce a latent variable  $z$  with simple prior  $p(z)$  (e.g. unit Gaussian)

Sample  $z \sim p(z)$  and pass to a **Generator Network**  $x = G(z)$

Then  $x$  is a sample from the **Generator distribution**  $p_G$ . Want  $p_G = p_{\text{data}}$ !



# Generative Adversarial Networks: Training Objective

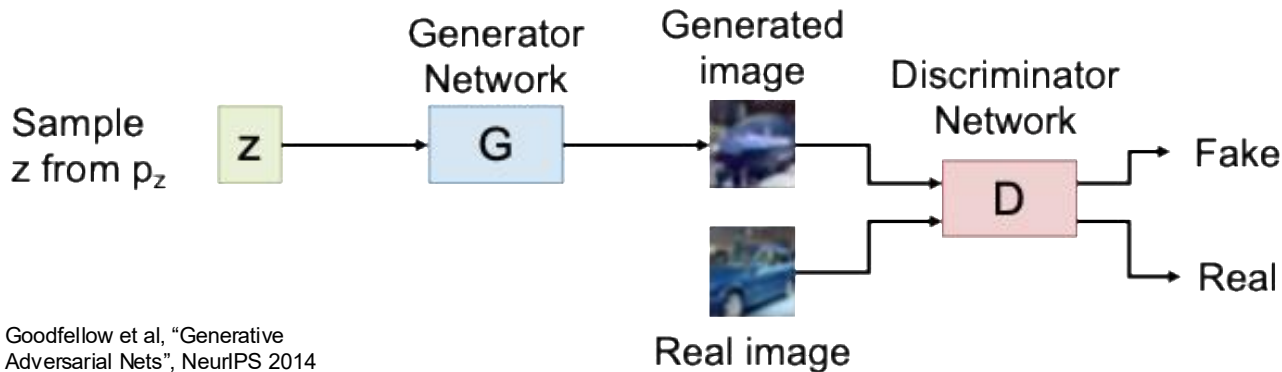
Jointly train generator  $G$  and discriminator  $D$  with a **minimax game**

$$\min_G \max_D \left( E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[ \log \left( 1 - D(G(z)) \right) \right] \right)$$

# Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_G \max_D \left( E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[ \log \left( 1 - D(G(z)) \right) \right] \right)$$



$D(x) = P(x \text{ is real})$   
 $D(x) = 0 \Rightarrow \text{fake}$   
 $D(x) = 1 \Rightarrow \text{real}$

# Generative Adversarial Networks: Training Objective

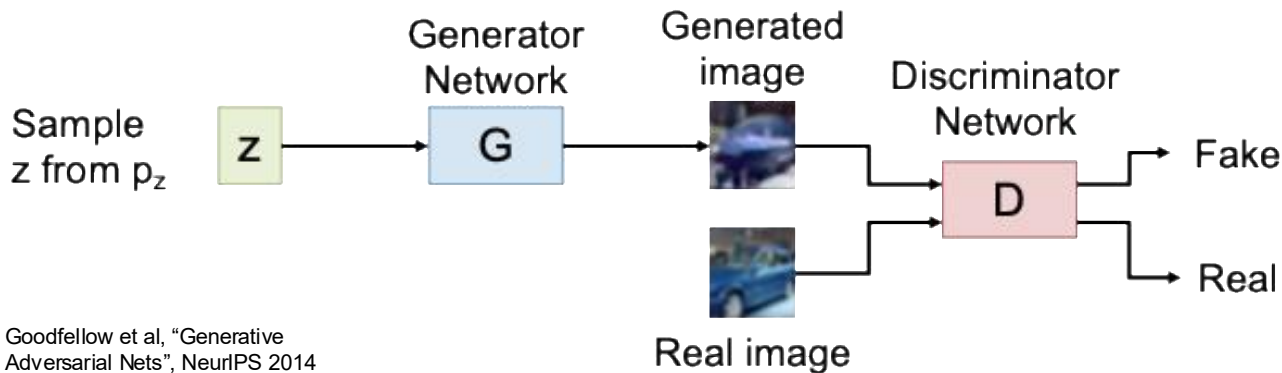
Jointly train generator G and discriminator D with a **minimax game**

Discriminator wants  
 $D(x) = 1$  for real data

Discriminator wants  
 $D(x) = 0$  for fake data

Imagine  
fixing **G**

$$\min_G \max_D \left( E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[ \log (1 - D(G(z))) \right] \right)$$



$D(x) = P(x \text{ is real})$   
 $D(x) = 0 \Rightarrow \text{fake}$   
 $D(x) = 1 \Rightarrow \text{real}$

# Generative Adversarial Networks: Training Objective

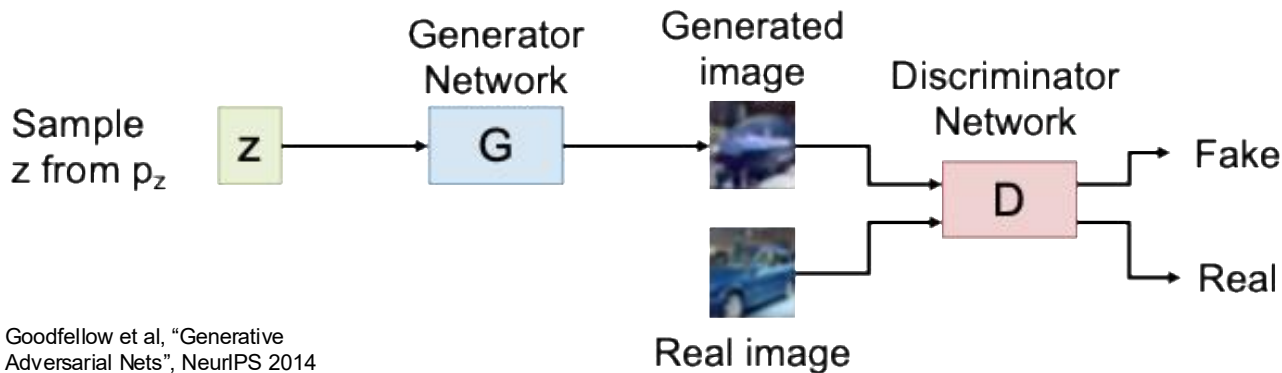
Jointly train generator G and discriminator D with a **minimax game**

Imagine fixing **D**

This term does not depend on G

Generator wants  $D(x) = 1$  for fake data

$$\min_G \max_D \left( E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[ \log \left( 1 - D(G(z)) \right) \right] \right)$$



$D(x) = P(x \text{ is real})$   
 $D(x) = 0 \Rightarrow \text{fake}$   
 $D(x) = 1 \Rightarrow \text{real}$

# Generative Adversarial Networks: Training Objective

Jointly train generator  $G$  and discriminator  $D$  with a **minimax game**

Train  $G$  and  $D$  using alternating gradient updates

$$\min_G \max_D \left( E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[ \log \left( 1 - D(G(z)) \right) \right] \right)$$
$$= \min_G \max_D V(G, D)$$

# Generative Adversarial Networks: Training Objective

Jointly train generator  $G$  and discriminator  $D$  with a **minimax game**

Train  $G$  and  $D$  using alternating gradient updates

$$\min_G \max_D \left( E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[ \log \left( 1 - D(G(z)) \right) \right] \right)$$

$$= \min_G \max_D V(G, D)$$

While True:

$$D = D + \alpha_D \frac{dV}{dD}$$

$$G = G - \alpha_G \frac{dV}{dG}$$

# Generative Adversarial Networks: Training Objective

Jointly train generator  $G$  and discriminator  $D$  with a **minimax game**

Train  $G$  and  $D$  using alternating gradient updates

$$\min_G \max_D \left( E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[ \log \left( 1 - D(G(z)) \right) \right] \right)$$

$$= \min_G \max_D V(G, D)$$

We are not minimizing any overall loss! No training curves to look at!

While True:

$$D = D + \alpha_D \frac{dV}{dD}$$

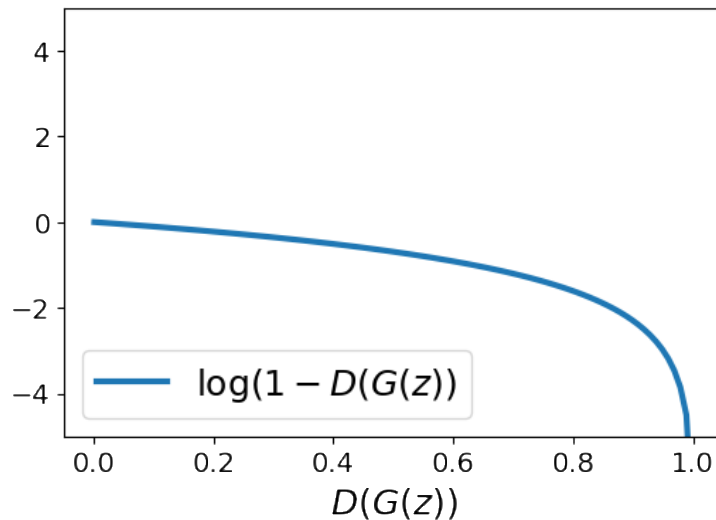
$$G = G - \alpha_G \frac{dV}{dG}$$

# Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left( E_{x \sim p_{data}} [\log \mathbf{D}(x)] + E_{z \sim p(z)} \left[ \log \left( 1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

At start of training, generator is very bad and discriminator can easily tell apart real/fake, so  $D(G(z))$  close to 0



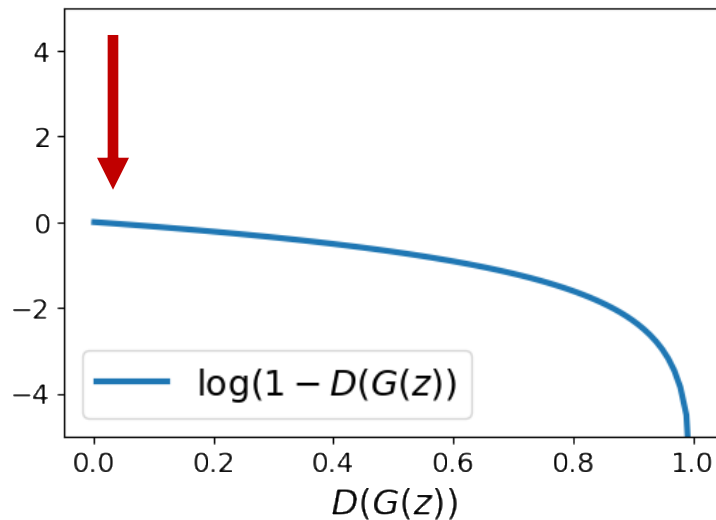
# Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_G \max_D \left( E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} [\log (1 - D(G(z)))] \right)$$

At start of training, generator is very bad and discriminator can easily tell apart real/fake, so  $D(G(z))$  close to 0

**Problem:** Gradients for G are close to 0



# Generative Adversarial Networks: Training Objective

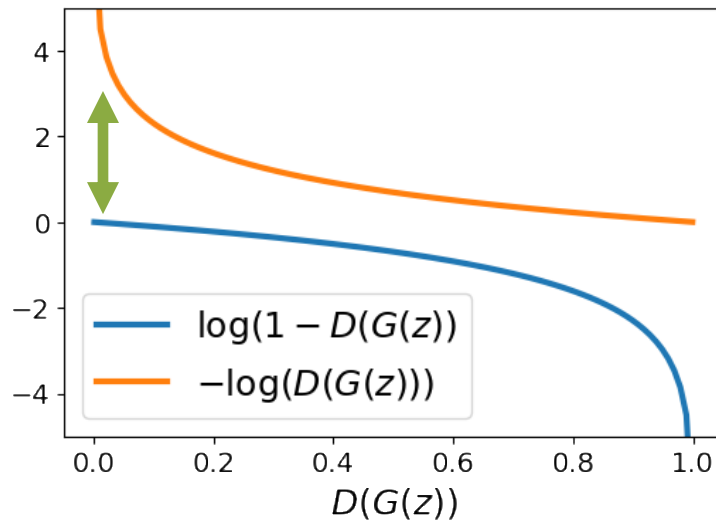
Jointly train generator G and discriminator D with a **minimax game**

$$\min_G \max_D \left( E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} [\log (1 - D(G(z)))] \right)$$

At start of training, generator is very bad and discriminator can easily tell apart real/fake, so  $D(G(z))$  close to 0

**Problem:** Gradients for G are close to 0

**Solution:** Generator wants  $D(G(z)) = 1$ .  
Train generator to minimize  $-\log(D(G(z)))$   
and discriminator to maximize  $\log(1-D(G(z)))$   
so generator gets strong gradients at start



# Generative Adversarial Networks: Training Objective

Jointly train generator  $G$  and discriminator  $D$  with a **minimax game**

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left( E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[ \log \left( 1 - D(G(z)) \right) \right] \right)$$

Why is this a good objective?

# Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left( E_{x \sim p_{data}} [\log \mathbf{D}(x)] + E_{z \sim p(z)} \left[ \log \left( 1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

Why is this a good objective?

Inner objective is maximized by

$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$$

(for any  $p_G$ )

(Proof omitted)

# Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left( E_{x \sim p_{data}} [\log \mathbf{D}(x)] + E_{z \sim p(z)} \left[ \log \left( 1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

Why is this a good objective?

Inner objective is maximized by

$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$$

(for any  $p_G$ )

Outer objective is  
then minimized by

$$p_G(x) = p_{data}(x)$$

(Proof omitted)

# Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left( E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[ \log \left( 1 - D(G(z)) \right) \right] \right)$$

Why is this a good objective?

Inner objective is maximized by

$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$$

(for any  $p_G$ )

(Proof omitted)

Outer objective is

then minimized by

$$p_G(x) = p_{data}(x)$$

Caveats:

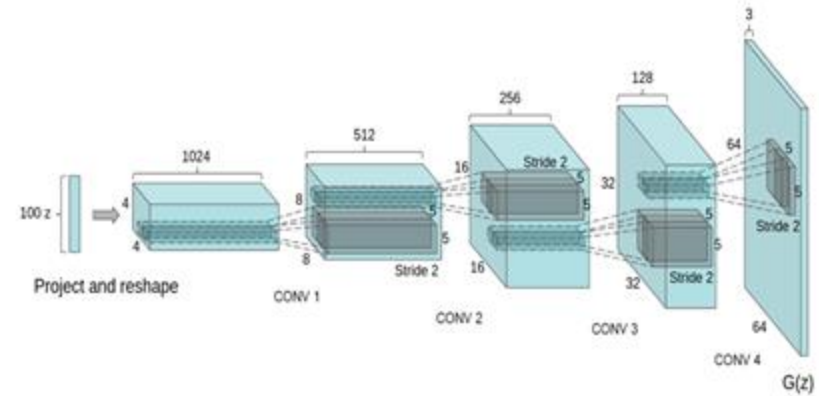
1. Neural nets with fixed capacity may not be able to represent optimal D and G
2. This tells us nothing about convergence to the solution with finite data

# GAN Architectures: DC-GAN

Generator G and discriminator D are both neural networks

Usually CNNs ... GANs fell out of favor before ViT became popular

DC-GAN was the first GAN architecture that worked on non-toy data



# GAN Architectures: DC-GAN

## GPT-1 Paper (2018)

Improving Language Understanding  
by Generative Pre-Training

Alec Radford  
OpenAI  
alec@openai.com

Karthik Narasimhan  
OpenAI  
karthikn@openai.com

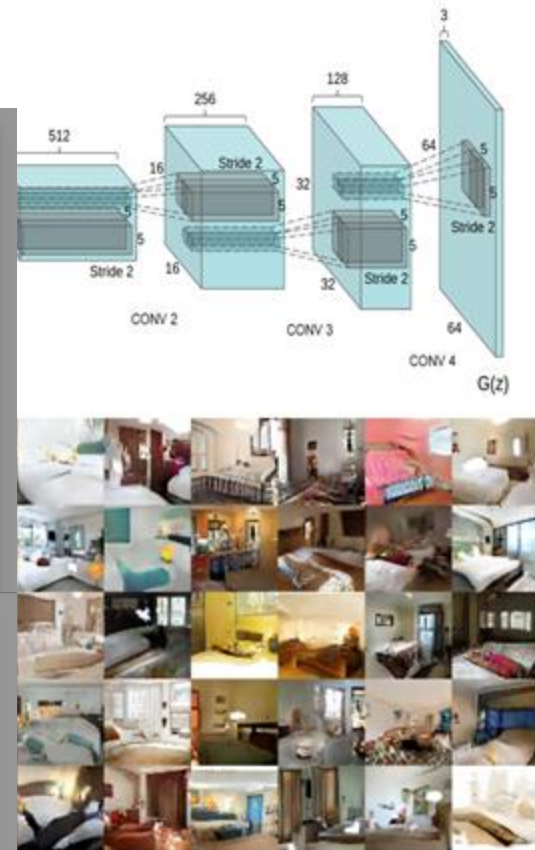
Tim Salimans  
OpenAI  
tim@openai.com

Ilya Sutskever  
OpenAI  
ilyasu@openai.com

## GPT-2 Paper (2019)

Language Models are Unsupervised Multitask Learners

Alec Radford \*<sup>1</sup> Jeffrey Wu \*<sup>1</sup> Rewon Child<sup>1</sup> David Luan<sup>1</sup> Dario Amodei \*\*<sup>1</sup> Ilya Sutskever \*\*<sup>1</sup>



Radford et al, ICLR 2016

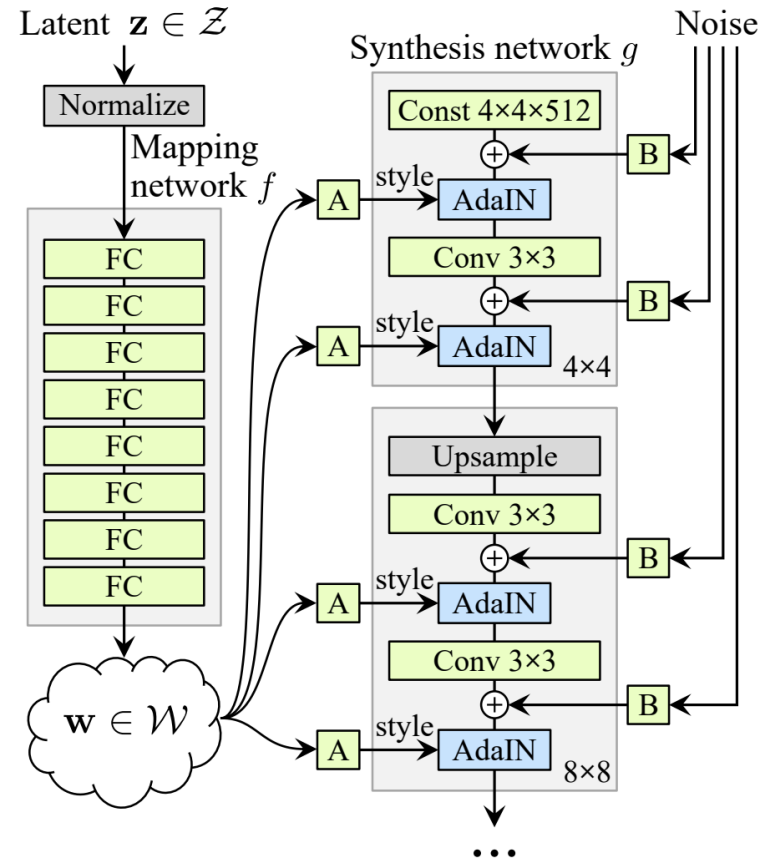
# GAN Architectures: StyleGAN

Generator G and discriminator D are both neural networks

StyleGAN uses a more complex architecture that injects noise via **adaptive normalization**.

At each layer predict a scale  $w$  and shift  $b$  the same shape as  $x$ :

$$AdaIN(x, w, b)_i = w_i \frac{x_i - \mu(x)}{\sigma(x)} + b_i$$



Karras et al, "A Style-Based Generator Architecture for Generative Adversarial Networks", CVPR 2019

# GANs: Latent Space Interpolation

Latent space is **smooth**.

Given latent vectors  $z_0$  and  $z_1$ , we can **interpolate** between them:

$$\begin{aligned}z_t &= tz_0 + (1 - t)z_1 \\x_t &= G(z_t)\end{aligned}$$

The resulting image  $x_t$  smoothly interpolate between samples!

# GANs: Latent Space Interpolation

Latent space is **smooth**.

Given latent vectors  $z_0$  and  $z_1$ , we can **interpolate** between them:

$$z_t = tz_0 + (1 - t)z_1$$
$$x_t = G(z_t)$$

The resulting image  $x_t$  smoothly interpolate between samples!



# Generative Adversarial Networks: Summary

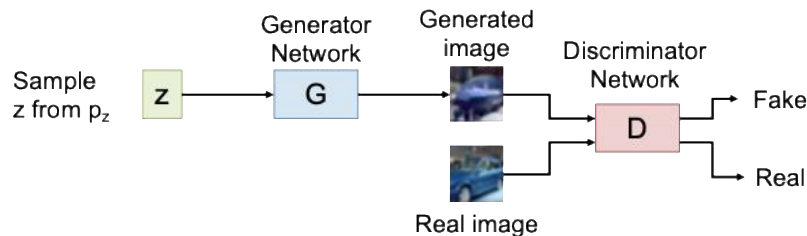
Jointly train Generator and Discriminator with a minimax game

## Pros:

- Simple formulation
- Very good image quality

## Cons:

- No loss curve to look at
- Unstable training
- Hard to scale to big models + data



These were the go-to generative models from ~2016 – 2021

# Diffusion Models

Sohl-Dickstein et al, "Deep Unsupervised Learning using nonequilibrium thermodynamics", ICML 2015  
Song and Ermon, "Generative modeling by estimating gradients of the data distribution", NeurIPS 2019  
Ho et al, "Denoising Diffusion Probabilistic Models", NeurIPS 2020  
Song et al, "Score-Based Generative Modeling through Stochastic Differential Equations", ICLR 2021  
Song et al, "Denoising Diffusion Implicit Models", ICLR 2021

# Diffusion Models

**Warning:** Terminology and notation in this area is a mess!

There are many different mathematical formalisms; tons of variance in terminology and notation between papers.

We'll just cover the basics of a modern “clean” implementation (Rectified Flow)

# Diffusion Models: Intuition

Pick a **noise distribution**  $z \sim p_{noise}$   
(Usually unit Gaussian)

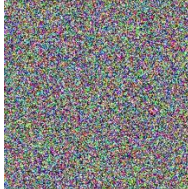
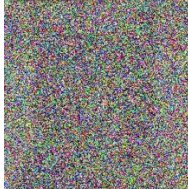
# Diffusion Models: Intuition

Pick a **noise distribution**  $z \sim p_{noise}$   
(Usually unit Gaussian)

Consider data  $x$  corrupted under varying  
**noise levels**  $t$  to give noisy data  $x_t$



$t = 0$   
No noise



$t = 1$   
Full noise

# Diffusion Models: Intuition

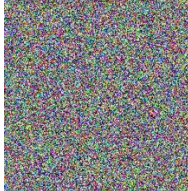
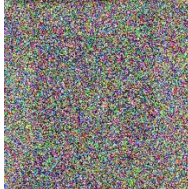
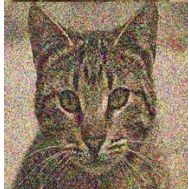
Pick a **noise distribution**  $z \sim p_{noise}$   
(Usually unit Gaussian)

Consider data  $x$  corrupted under varying **noise levels**  $t$  to give noisy data  $x_t$

Train a neural network to **remove a little bit of noise**:  $f_{\theta}(x_t, t)$



$t = 0$   
No noise



$t = 1$   
Full noise

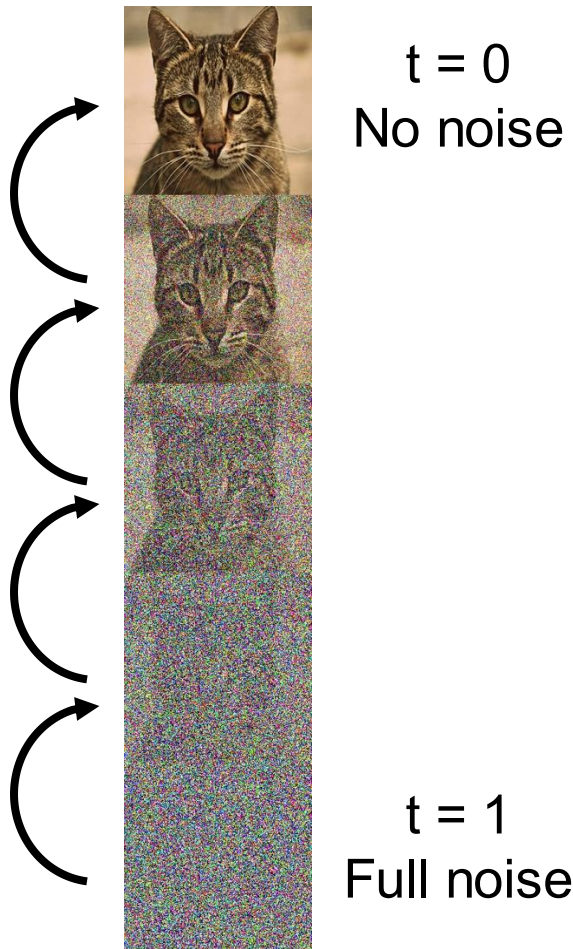
# Diffusion Models: Intuition

Pick a **noise distribution**  $z \sim p_{noise}$   
(Usually unit Gaussian)

Consider data  $x$  corrupted under varying **noise levels**  $t$  to give noisy data  $x_t$

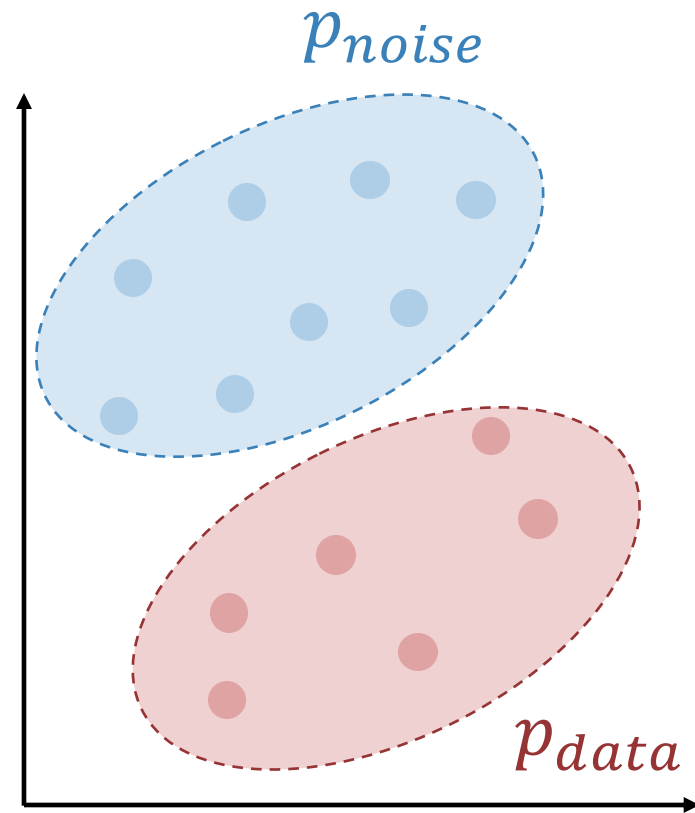
Train a neural network to **remove a little bit of noise**:  $f_{\theta}(x_t, t)$

At inference time, sample  $x_1 \sim p_{noise}$  and apply  $f_{\theta}$  many times in sequence to generate a noiseless sample  $x_0$



# Diffusion Models: Rectified Flow

Suppose we have a simple  $p_{\text{noise}}$  (e.g. Gaussian) and samples from  $p_{\text{data}}$

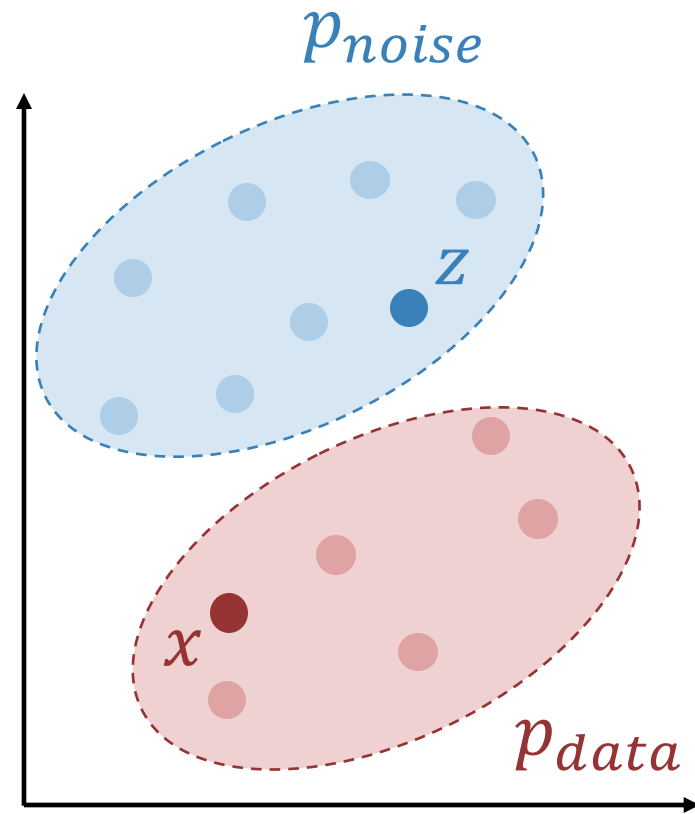


# Diffusion Models: Rectified Flow

Suppose we have a simple  $p_{\text{noise}}$  (e.g. Gaussian) and samples from  $p_{\text{data}}$

On each training iteration, sample:

$$z \sim p_{\text{noise}} \quad x \sim p_{\text{data}} \quad t \sim \text{Uniform}[0, 1]$$



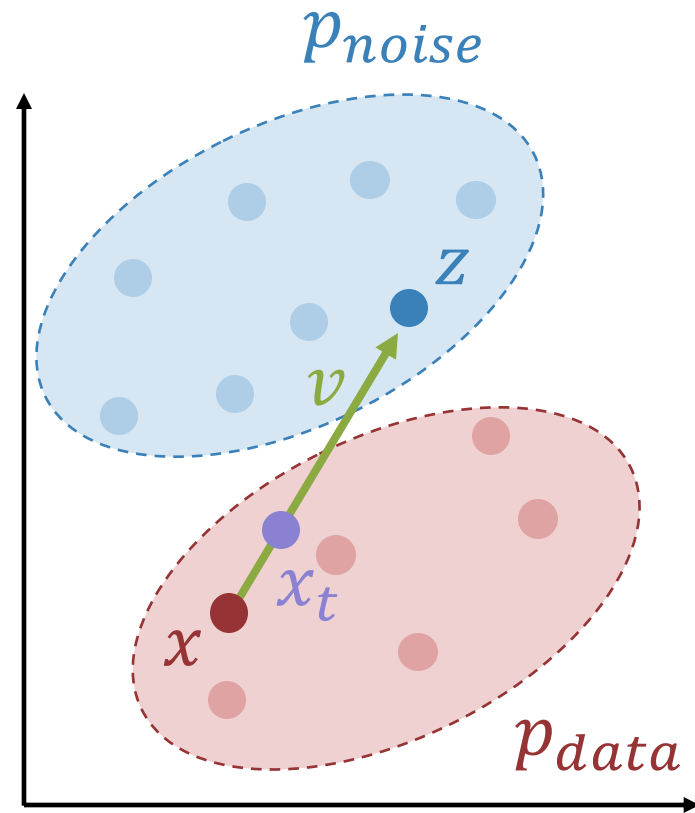
# Diffusion Models: Rectified Flow

Suppose we have a simple  $p_{\text{noise}}$  (e.g. Gaussian) and samples from  $p_{\text{data}}$

On each training iteration, sample:

$$z \sim p_{\text{noise}} \quad x \sim p_{\text{data}} \quad t \sim \text{Uniform}[0, 1]$$

Set  $x_t = (1 - t)x + tz$ ,  $v = z - x$



# Diffusion Models: Rectified Flow

Suppose we have a simple  $p_{\text{noise}}$  (e.g. Gaussian) and samples from  $p_{\text{data}}$

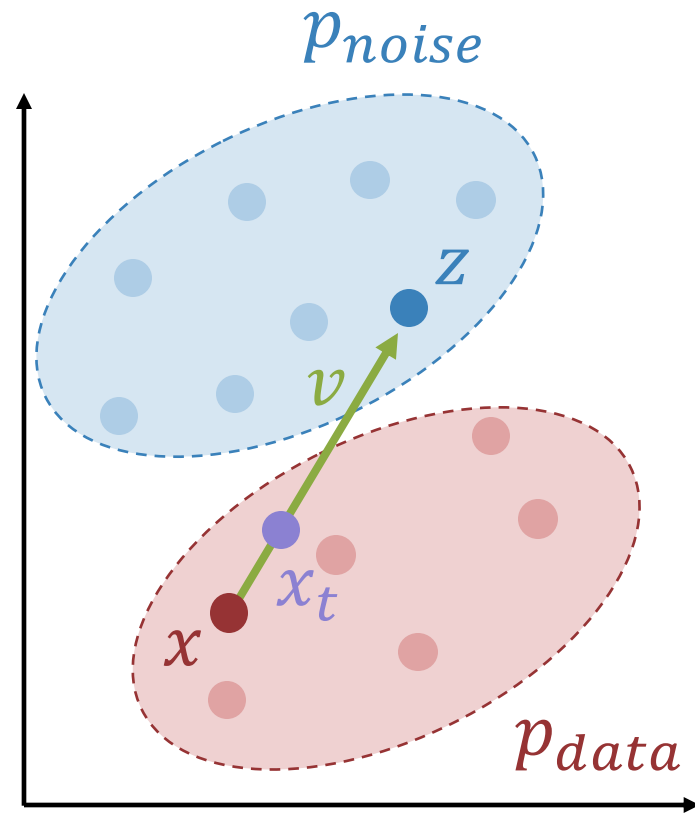
On each training iteration, sample:

$$z \sim p_{\text{noise}} \quad x \sim p_{\text{data}} \quad t \sim \text{Uniform}[0, 1]$$

Set  $x_t = (1 - t)x + tz$ ,  $v = z - x$

Train a neural network to predict  $v$ :

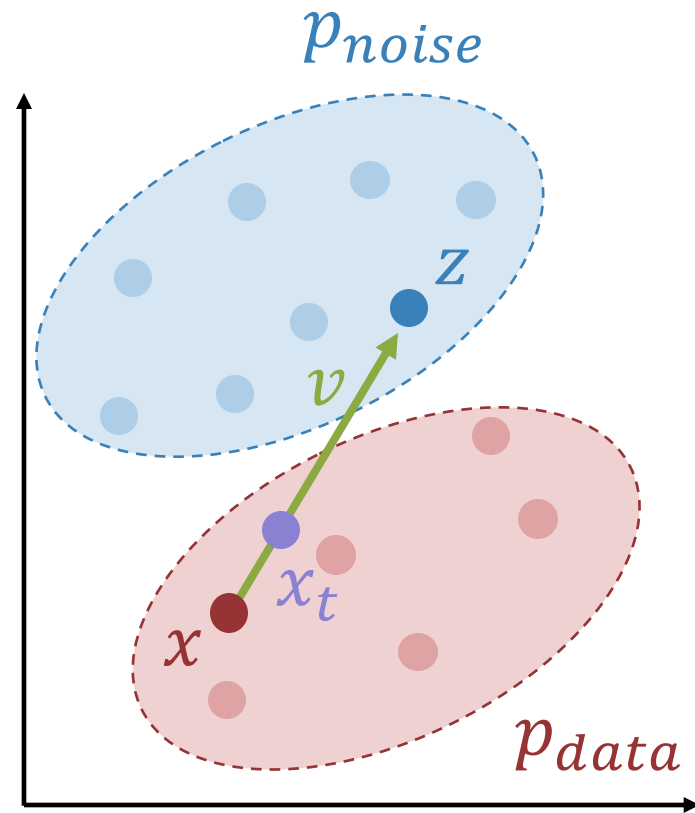
$$L = \|f_{\theta}(x_t, t) - v\|_2^2$$



# Diffusion Models: Rectified Flow

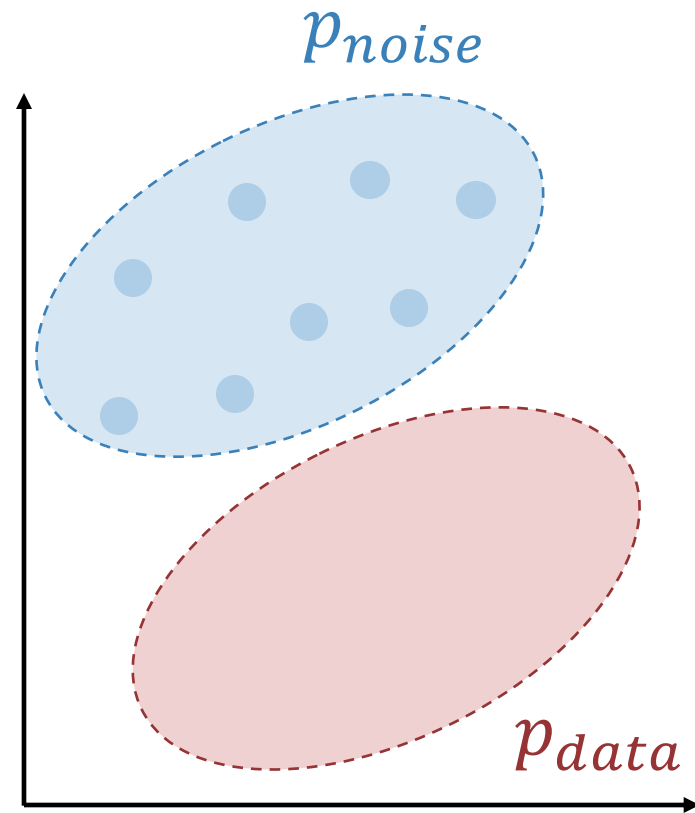
Core training loop is just  
a few lines of code!

```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```



# Rectified Flow: Sampling

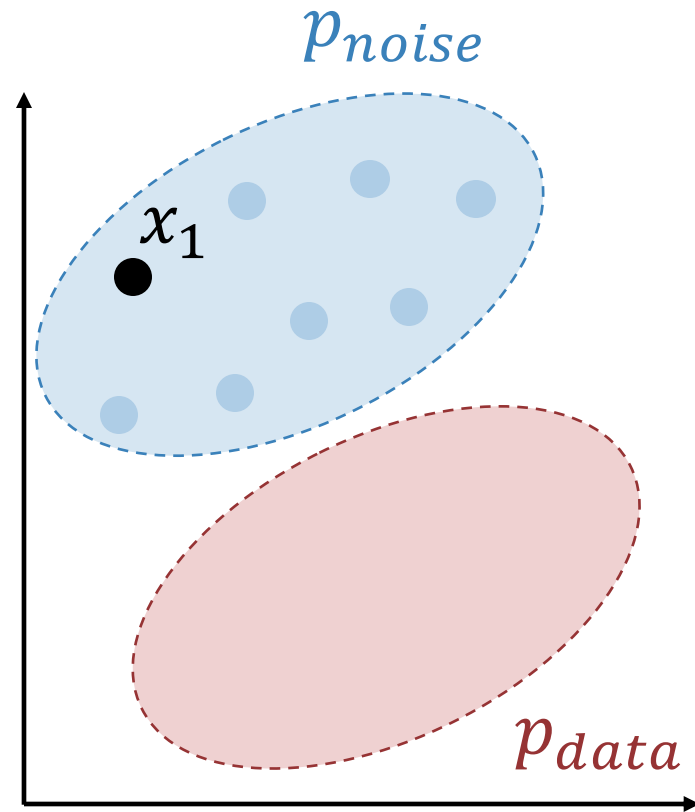
Choose number of steps  $T$  (often  $T=50$ )



# Rectified Flow: Sampling

Choose number of steps  $T$  (often  $T=50$ )

Sample  $x \sim p_{noise}$



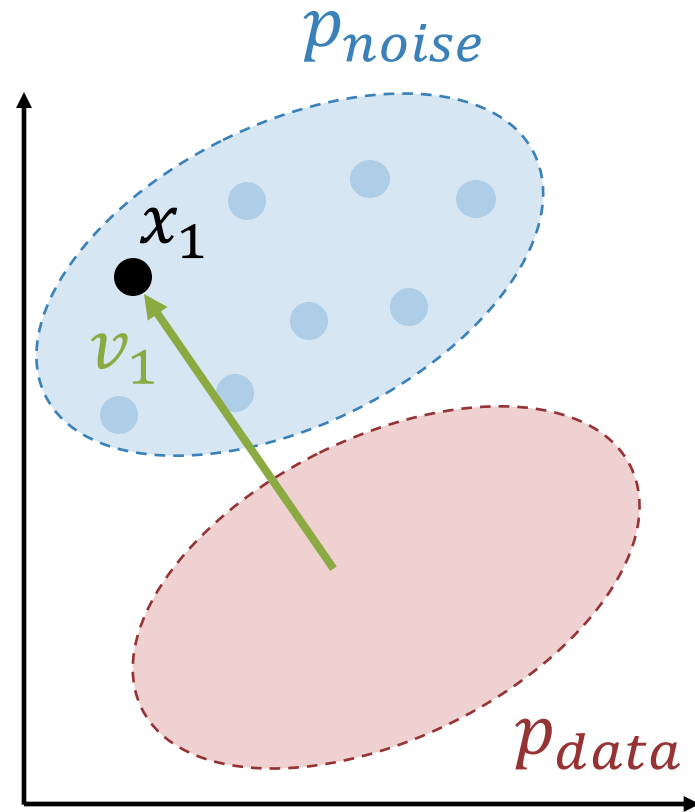
# Rectified Flow: Sampling

Choose number of steps  $T$  (often  $T=50$ )

Sample  $x \sim p_{noise}$

For  $t$  in  $[1, 1 - \frac{1}{T}, 1 - \frac{2}{T}, \dots, 0]$ :

Evaluate  $v_t = f_\theta(x_t, t)$



# Rectified Flow: Sampling

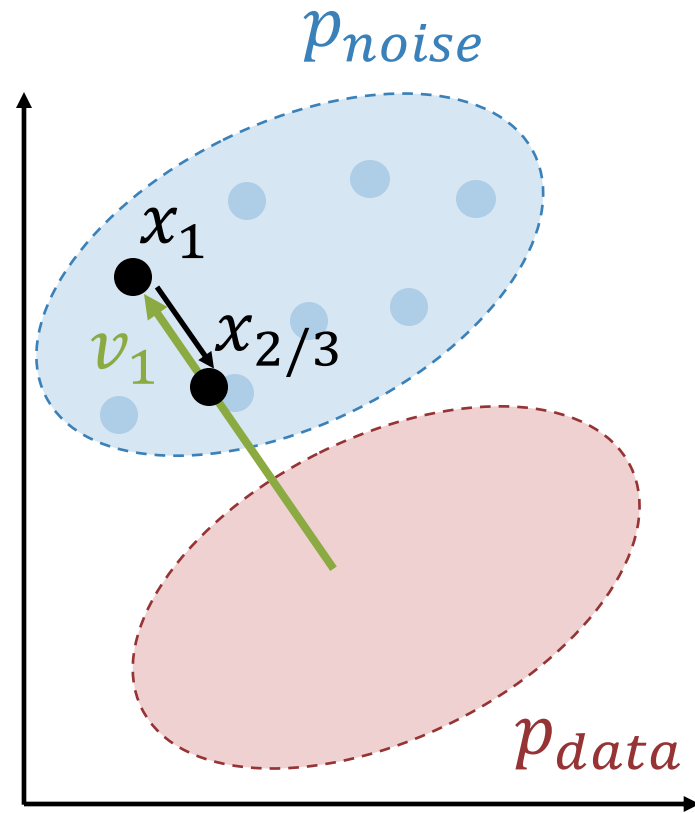
Choose number of steps  $T$  (often  $T=50$ )

Sample  $x \sim p_{noise}$

For  $t$  in  $[1, 1 - \frac{1}{T}, 1 - \frac{2}{T}, \dots, 0]$ :

Evaluate  $v_t = f_\theta(x_t, t)$

Step  $x = x - v_t/T$



# Rectified Flow: Sampling

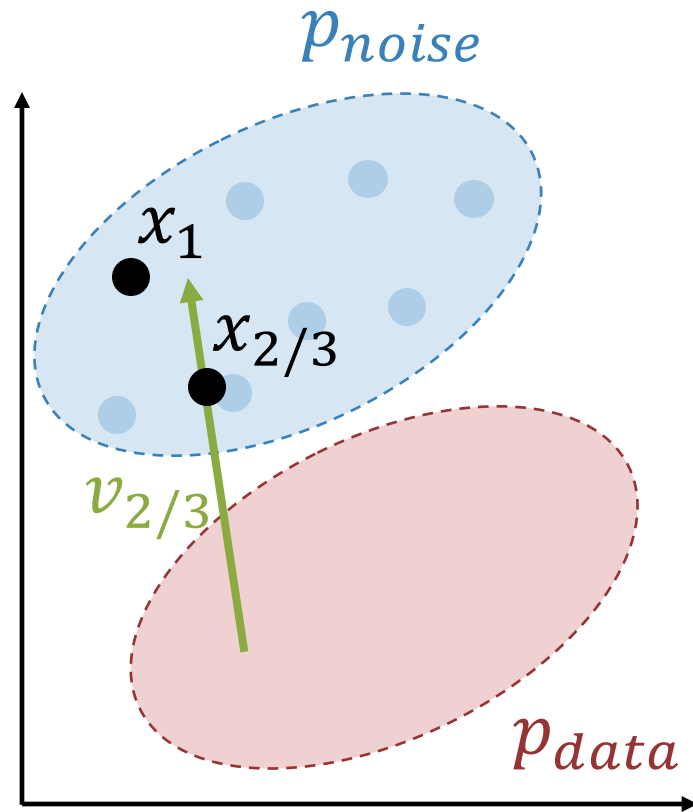
Choose number of steps  $T$  (often  $T=50$ )

Sample  $x \sim p_{noise}$

For  $t$  in  $[1, 1 - \frac{1}{T}, 1 - \frac{2}{T}, \dots, 0]$ :

Evaluate  $v_t = f_\theta(x_t, t)$

Step  $x = x - v_t/T$



# Rectified Flow: Sampling

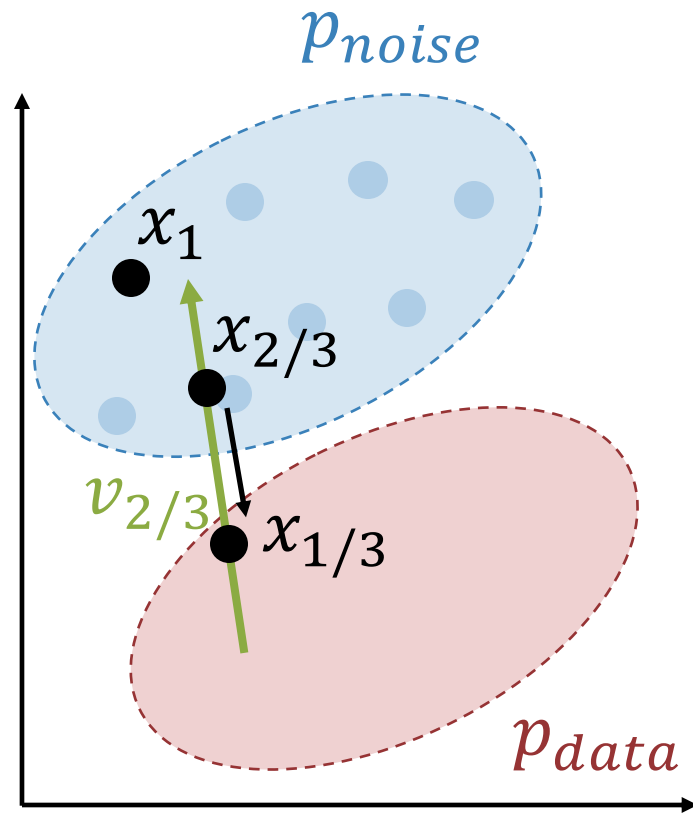
Choose number of steps  $T$  (often  $T=50$ )

Sample  $x \sim p_{noise}$

For  $t$  in  $[1, 1 - \frac{1}{T}, 1 - \frac{2}{T}, \dots, 0]$ :

Evaluate  $v_t = f_\theta(x_t, t)$

Step  $x = x - v_t/T$



# Rectified Flow: Sampling

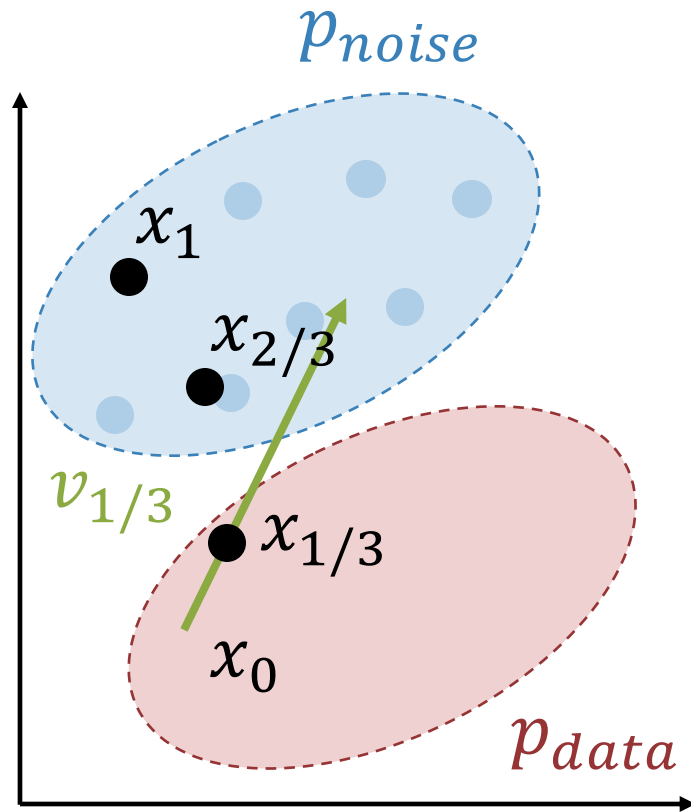
Choose number of steps  $T$  (often  $T=50$ )

Sample  $x \sim p_{noise}$

For  $t$  in  $[1, 1 - \frac{1}{T}, 1 - \frac{2}{T}, \dots, 0]$ :

Evaluate  $v_t = f_\theta(x_t, t)$

Step  $x = x - v_t/T$



# Rectified Flow: Sampling

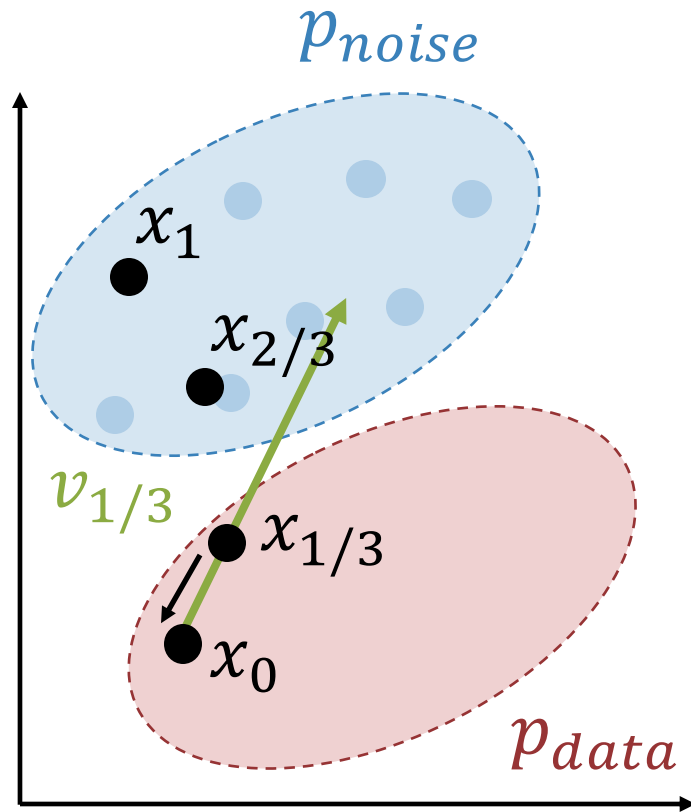
Choose number of steps  $T$  (often  $T=50$ )

Sample  $x \sim p_{noise}$

For  $t$  in  $[1, 1 - \frac{1}{T}, 1 - \frac{2}{T}, \dots, 0]$ :

Evaluate  $v_t = f_\theta(x_t, t)$

Step  $x = x - v_t/T$



# Rectified Flow: Sampling

Choose number of steps  $T$  (often  $T=50$ )

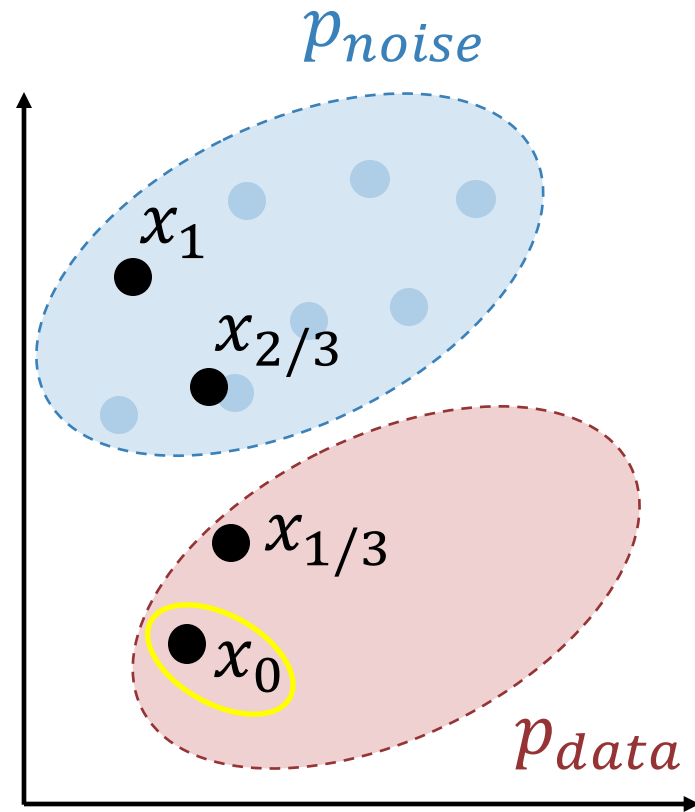
Sample  $x \sim p_{noise}$

For  $t$  in  $[1, 1 - \frac{1}{T}, 1 - \frac{2}{T}, \dots, 0]$ :

Evaluate  $v_t = f_\theta(x_t, t)$

Step  $x = x - v_t/T$

Return  $x$



# Rectified Flow: Sampling

Choose number of steps  $T$  (often  $T=50$ )

Sample  $x \sim p_{noise}$

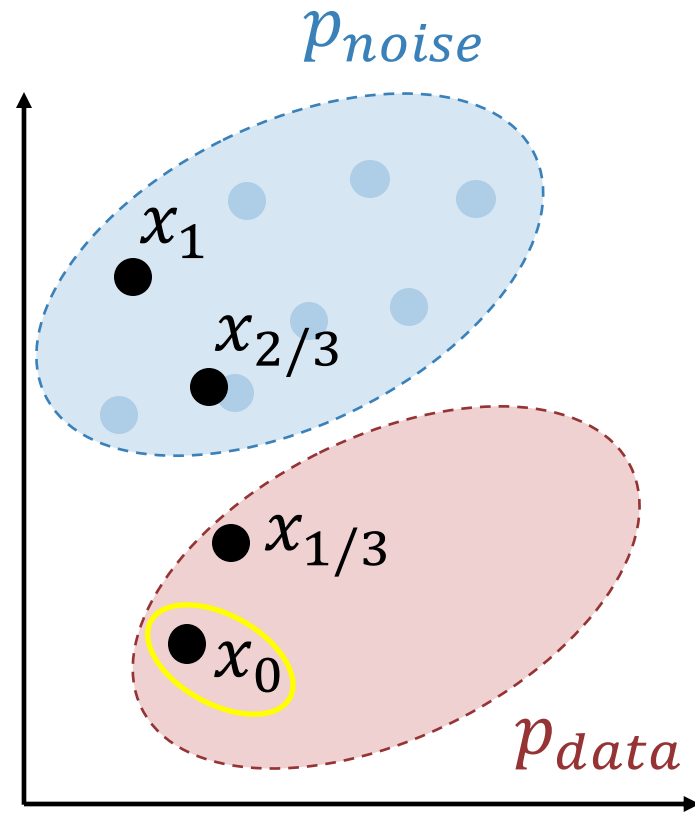
For  $t$  in  $[1, 1 - \frac{1}{T}, 1 - \frac{2}{T}, \dots, 0]$ :

Evaluate  $v_t = f_\theta(x_t, t)$

Step  $x = x - v_t/T$

Return  $x$

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```



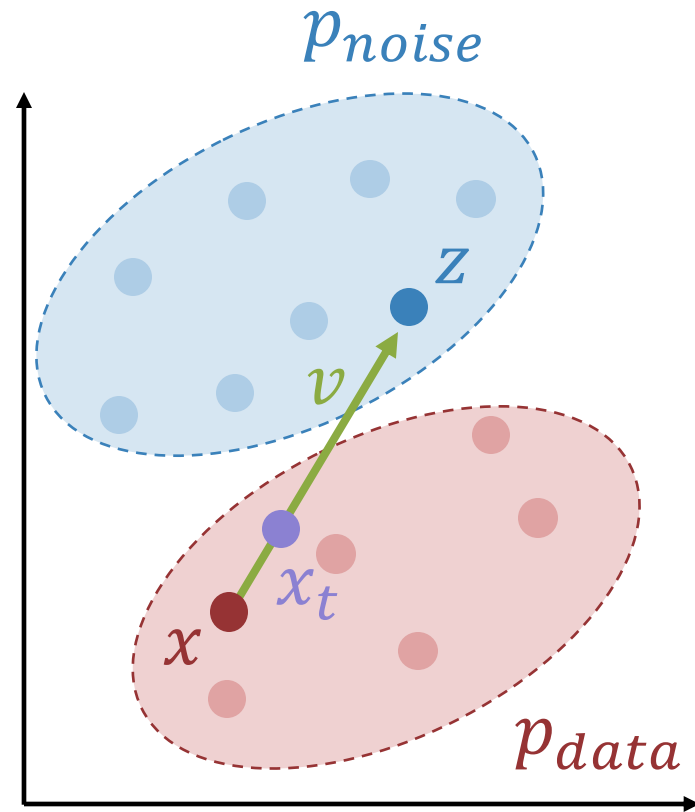
# Rectified Flow: Summary

## Training

```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```

## Sampling

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```



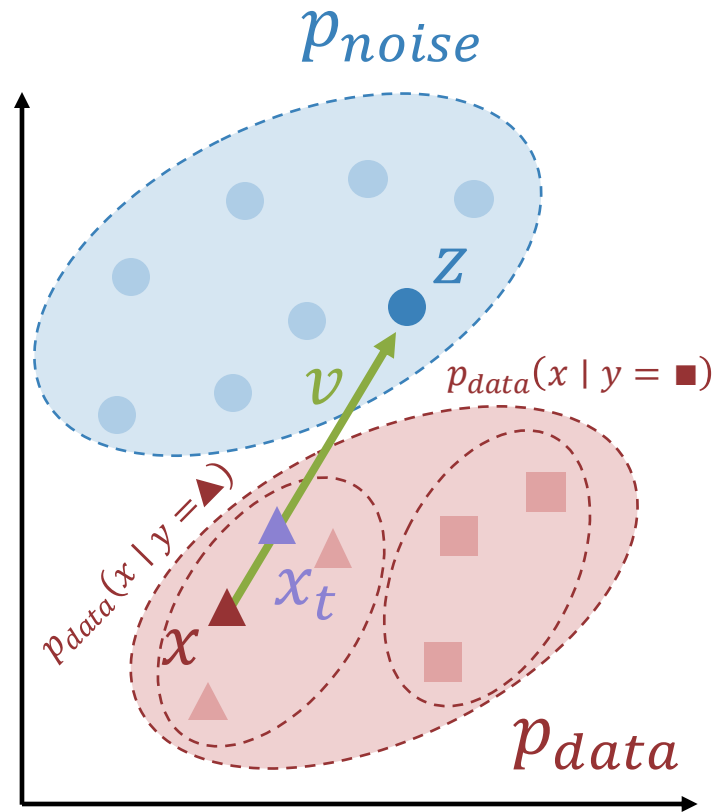
# Conditional Rectified Flow

## Training

```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```

## Sampling

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```



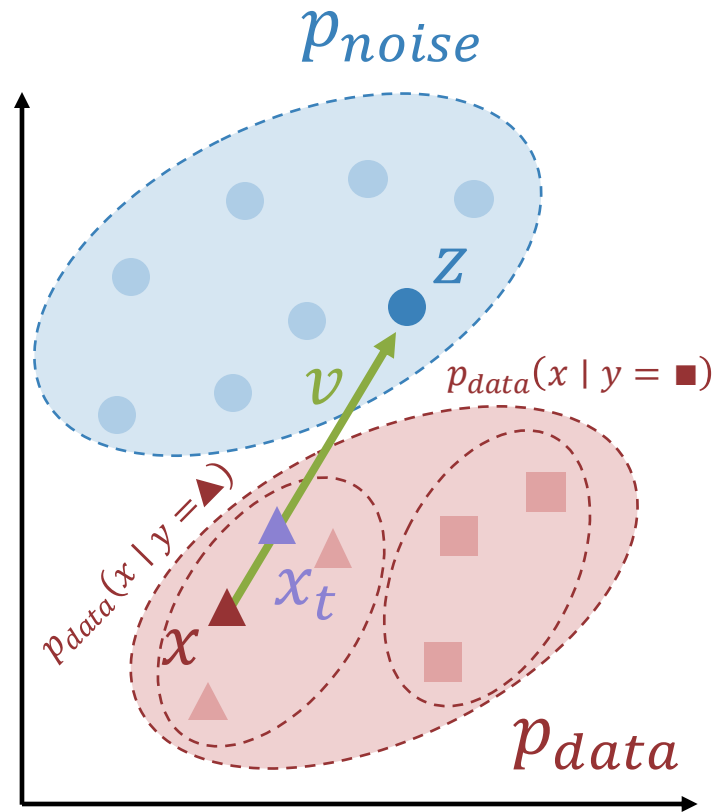
# Conditional Rectified Flow

## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

## Sampling

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```



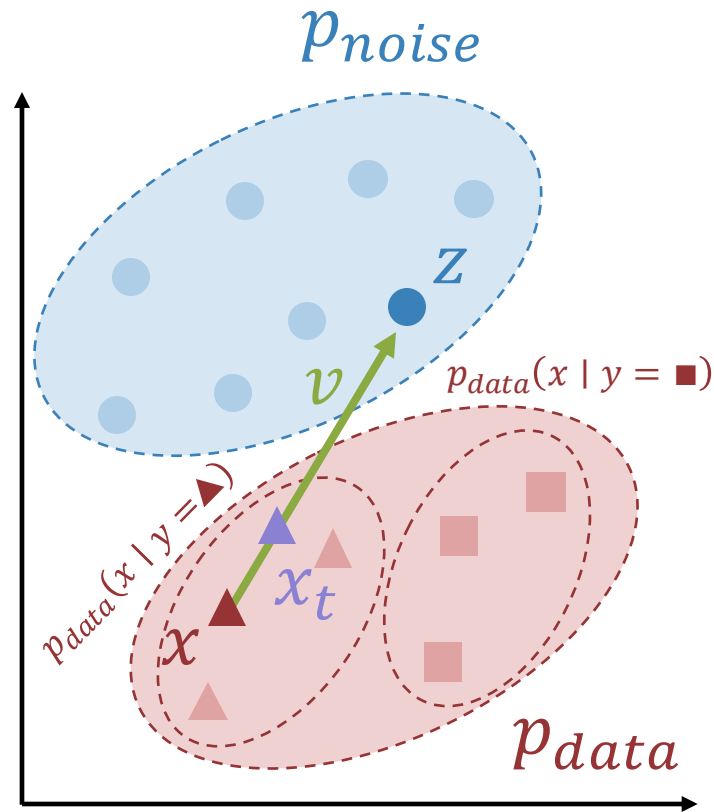
# Conditional Rectified Flow

## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

## Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, y, t)
    sample = sample - v / num_steps
```



# Conditional Rectified Flow

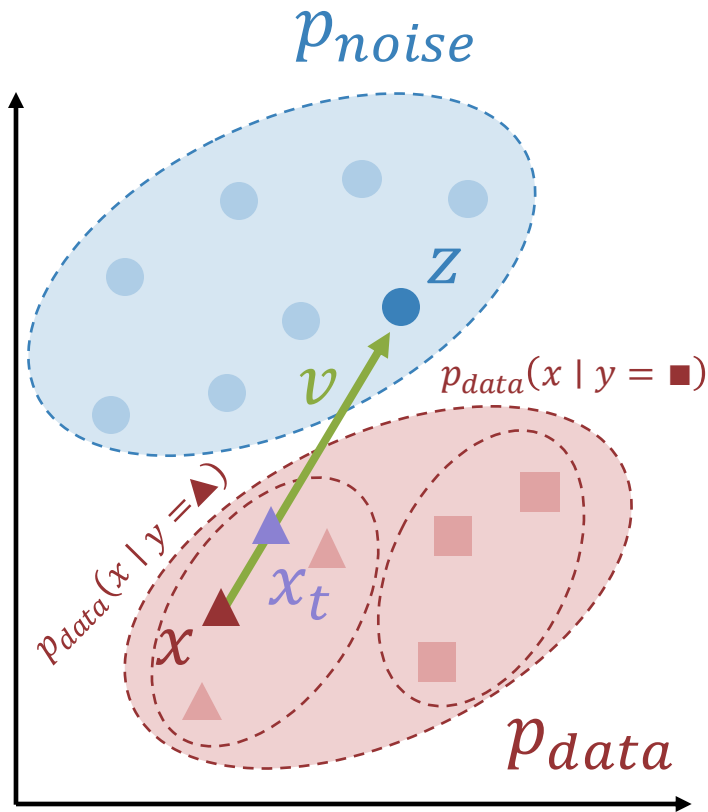
## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we “emphasize” the conditioning  $y$ ?

## Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, y, t)
    sample = sample - v / num_steps
```



# Classifier-Free Guidance (CFG)

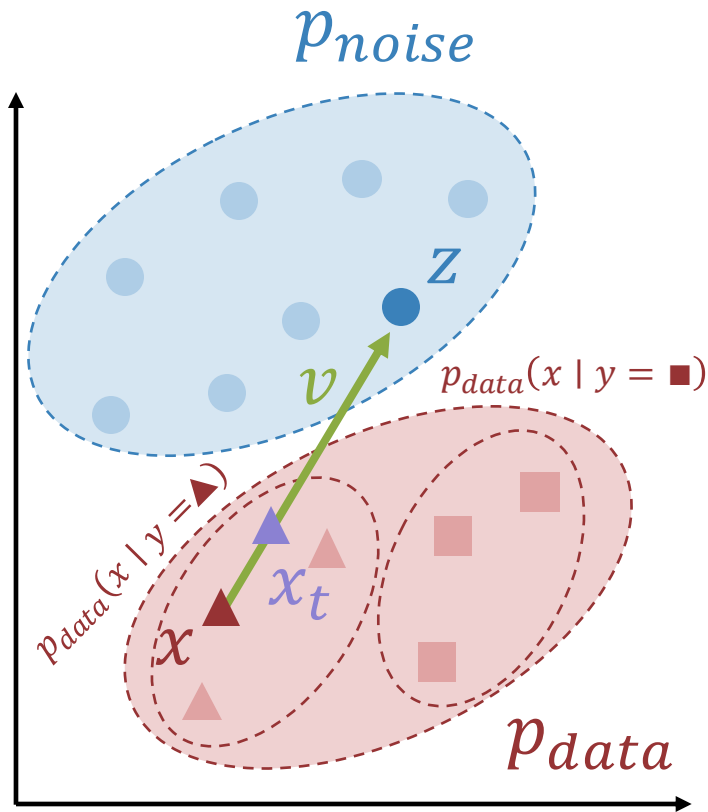
## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we “emphasize” the conditioning  $y$ ?

Randomly drop  $y$  during training.

Now the same model is conditional and unconditional!



# Classifier-Free Guidance (CFG)

## Training

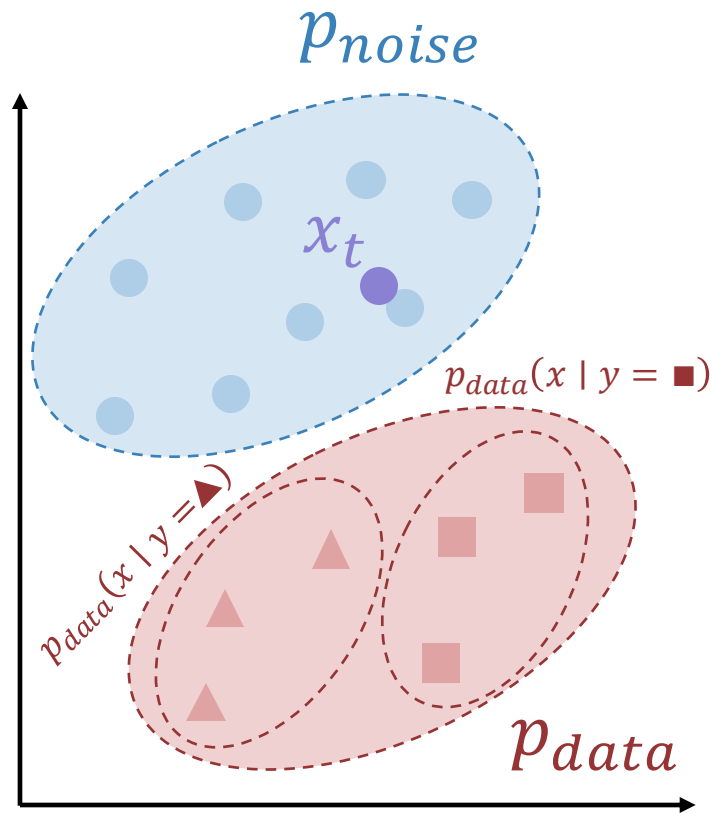
```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we “emphasize” the conditioning  $y$ ?

Randomly drop  $y$  during training.

Now the same model is conditional and unconditional!

Consider a noisy  $x_t$ :



# Classifier-Free Guidance (CFG)

## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

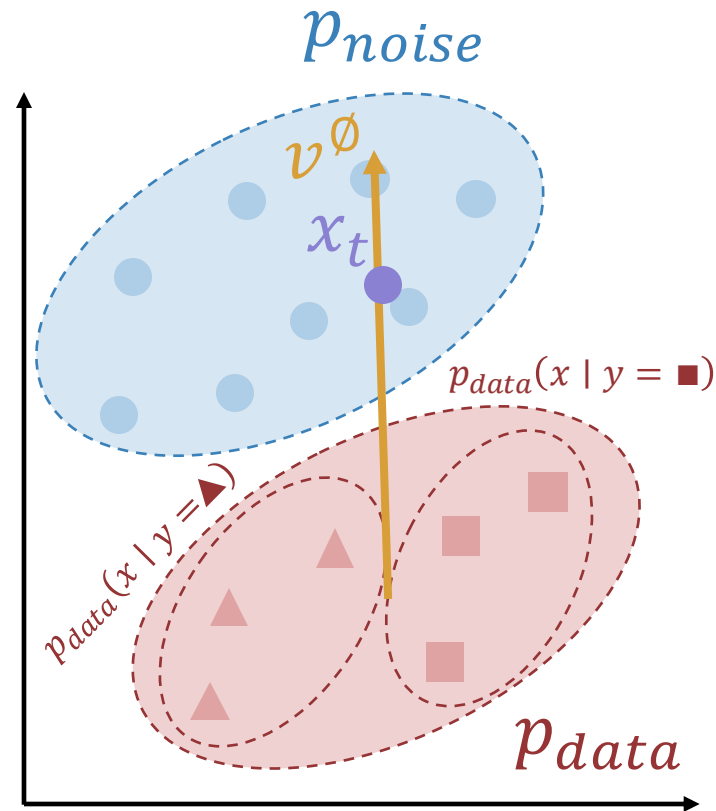
Can we control how much we “emphasize” the conditioning  $y$ ?

Randomly drop  $y$  during training.

Now the same model is conditional and unconditional!

Consider a noisy  $x_t$ :

$v^\emptyset = f_\theta(x_t, y_\emptyset, t)$  points toward  $p(x)$



# Classifier-Free Guidance (CFG)

## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we “emphasize” the conditioning  $y$ ?

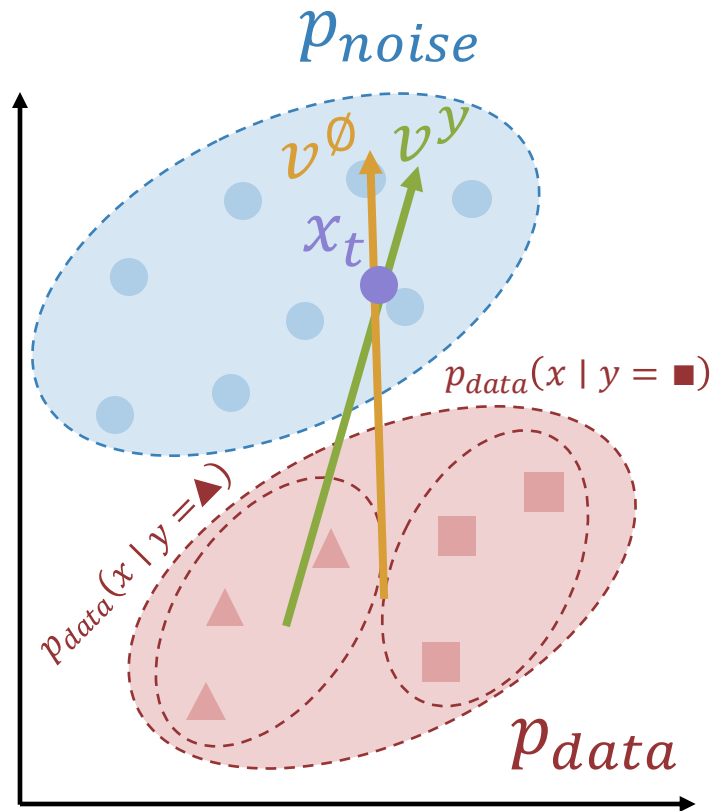
Randomly drop  $y$  during training.

Now the same model is conditional and unconditional!

Consider a noisy  $x_t$ :

$v^\emptyset = f_\theta(x_t, y_\emptyset, t)$  points toward  $p(x)$

$v^y = f_\theta(x_t, y, t)$  points toward  $p(x | y)$



# Classifier-Free Guidance (CFG)

## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we “emphasize” the conditioning  $y$ ?

Randomly drop  $y$  during training.

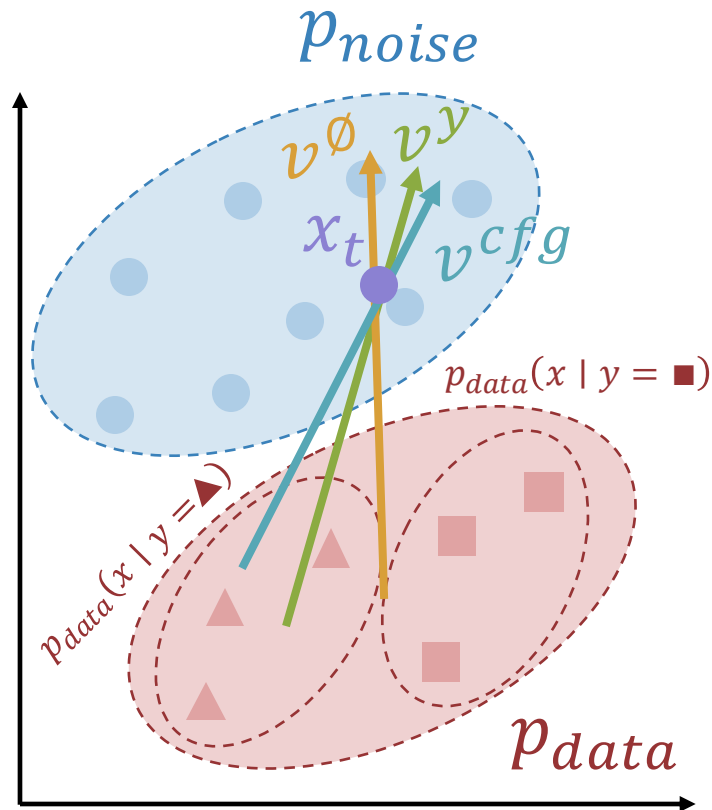
Now the same model is conditional and unconditional!

Consider a noisy  $x_t$ :

$v^\emptyset = f_\theta(x_t, y_\emptyset, t)$  points toward  $p(x)$

$v^y = f_\theta(x_t, y, t)$  points toward  $p(x | y)$

$v^{cfg} = (1 + w)v^y - wv^\emptyset$  points more toward  $p(x | y)$



# Classifier-Free Guidance (CFG)

## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we “emphasize” the conditioning  $y$ ?

Randomly drop  $y$  during training.

Now the same model is conditional and unconditional!

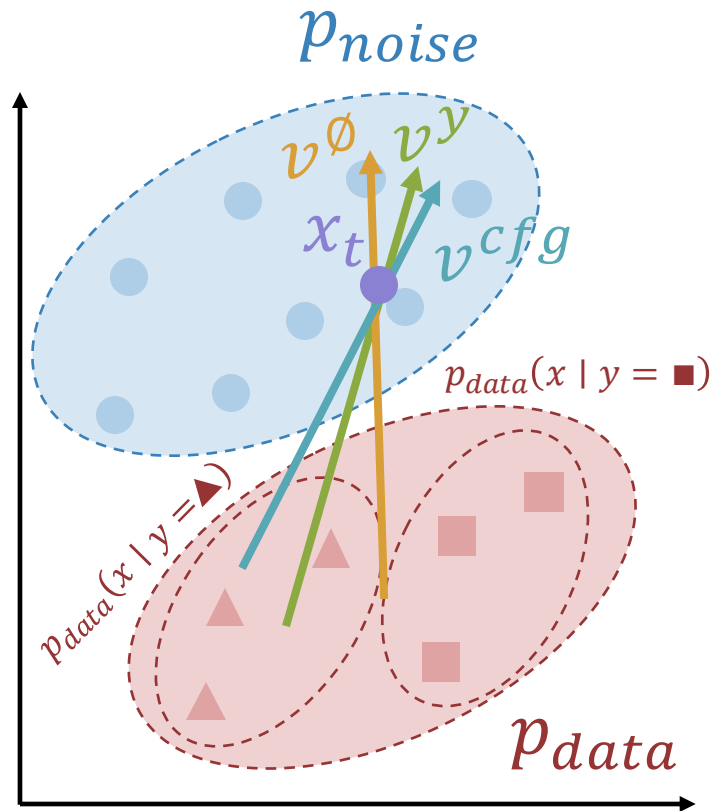
Consider a noisy  $x_t$ :

$v^\emptyset = f_\theta(x_t, y_\emptyset, t)$  points toward  $p(x)$

$v^y = f_\theta(x_t, y, t)$  points toward  $p(x | y)$

$v^{cfg} = (1 + w)v^y - wv^\emptyset$  points more toward  $p(x | y)$

During sampling, step according to  $v^{cfg}$



# Classifier-Free Guidance (CFG)

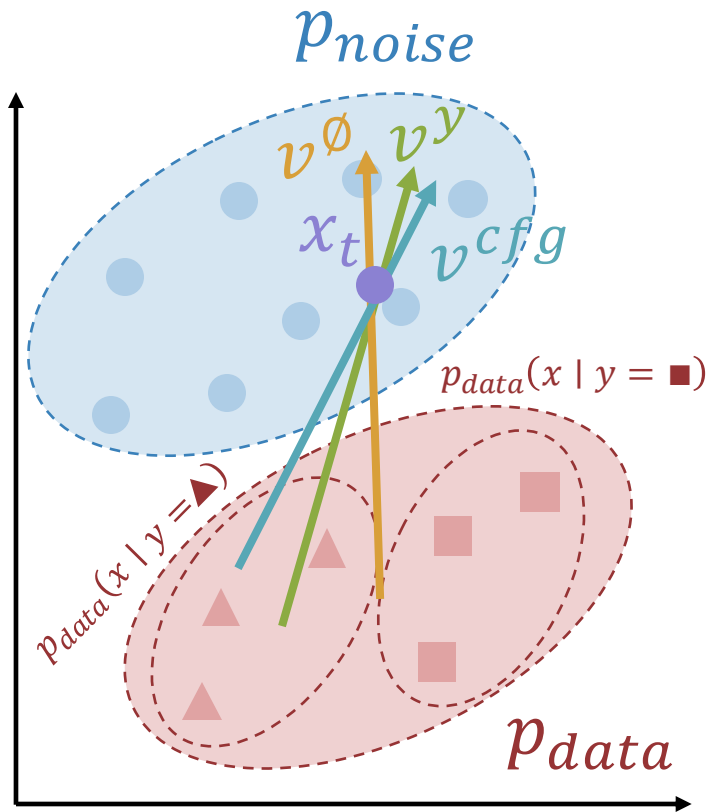
## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we “emphasize” the conditioning  $y$ ?

## Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    vy = model(sample, y, t)
    v0 = model(sample, y_null, t)
    v = (1 + w) * vy - w * v0
    sample = sample - v / num_steps
```



# Classifier-Free Guidance (CFG)

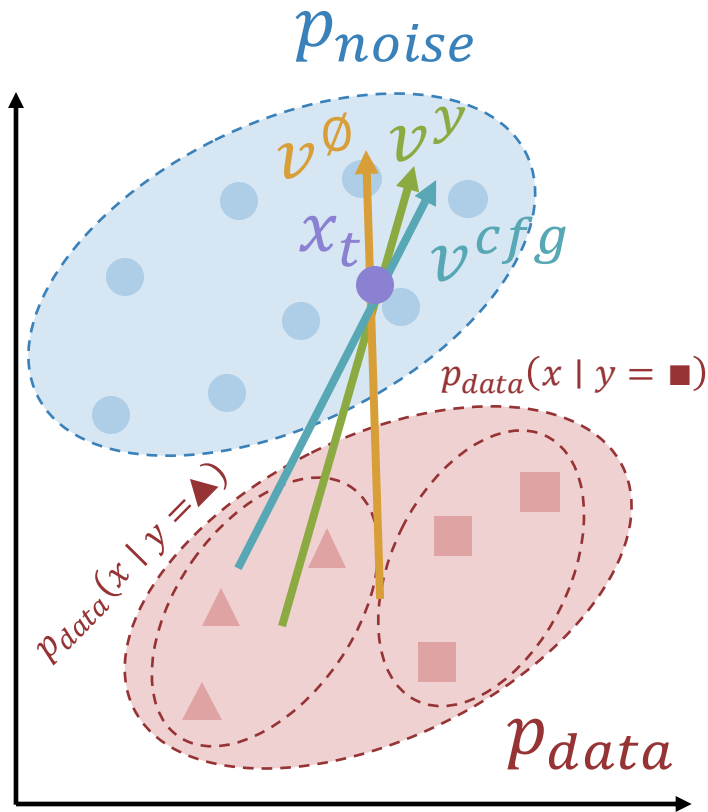
## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we “emphasize” the conditioning  $y$ ?

## Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    vy = model(sample, y, t)
    v0 = model(sample, y_null, t)
    v = (1 + w) * vy - w * v0
    sample = sample - v / num_steps
```



# Classifier-Free Guidance (CFG)

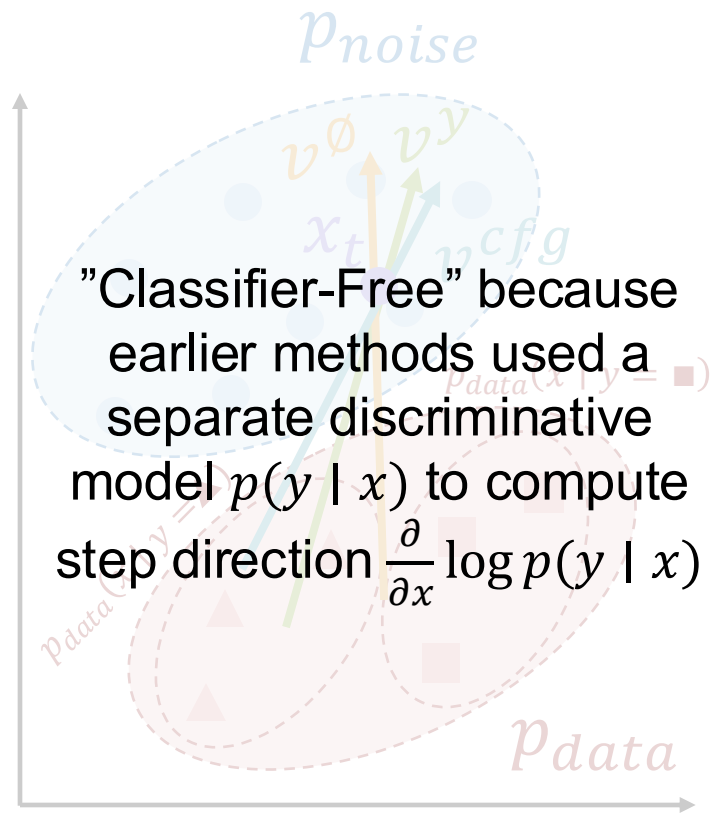
## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we “emphasize” the conditioning  $y$ ?

## Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    vy = model(sample, y, t)
    v0 = model(sample, y_null, t)
    v = (1 + w) * vy - w * v0
    sample = sample - v / num_steps
```



# Classifier-Free Guidance (CFG)

## Training

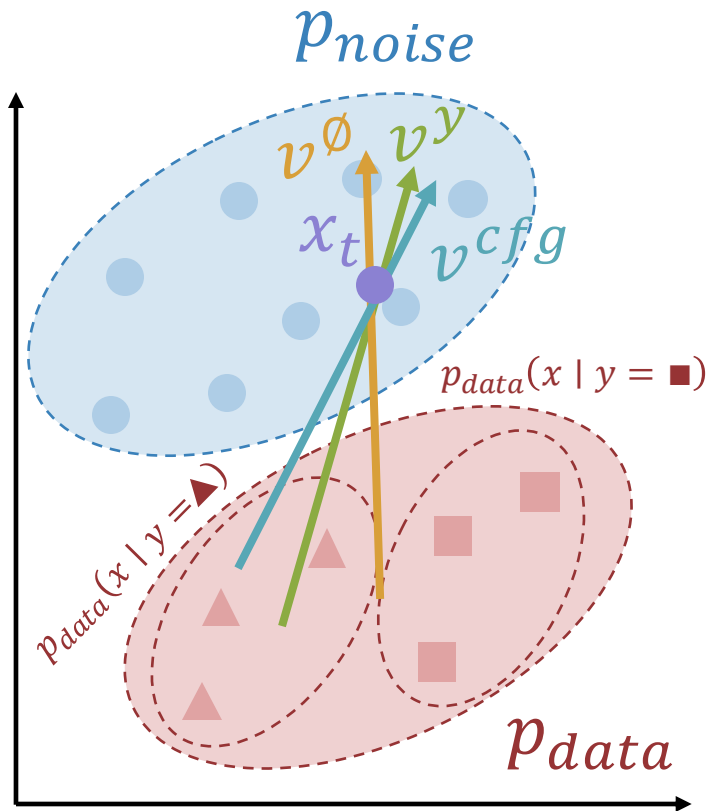
```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we “emphasize” the conditioning  $y$ ?

## Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    vy = model(sample, y, t)
    v0 = model(sample, y_null, t)
    v = (1 + w) * vy - w * v0
    sample = sample - v / num_steps
```

Used everywhere in practice! Very important for high-quality outputs



# Classifier-Free Guidance (CFG)

## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

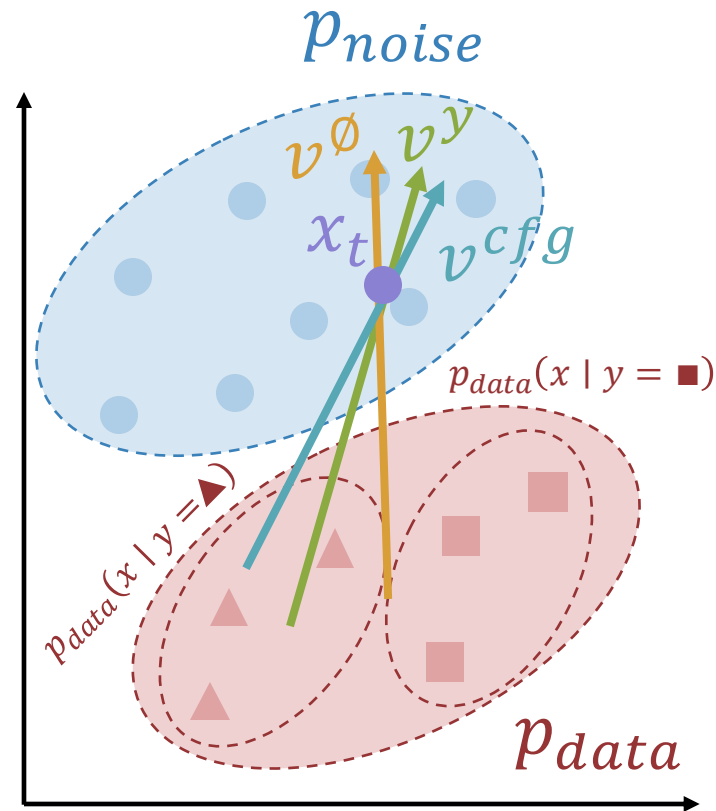
Can we control how much we “emphasize” the conditioning  $y$ ?

## Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    vy = model(sample, y, t)
    v0 = model(sample, y_null, t)
    v = (1 + w) * vy - w * v0
    sample = sample - v / num_steps
```

Used everywhere in practice! Very important for high-quality outputs

Doubles the cost of sampling...



# Noise Schedules

## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

There may be many pairs  $(x, z)$  that give the same  $x_t$ ; network must average over them

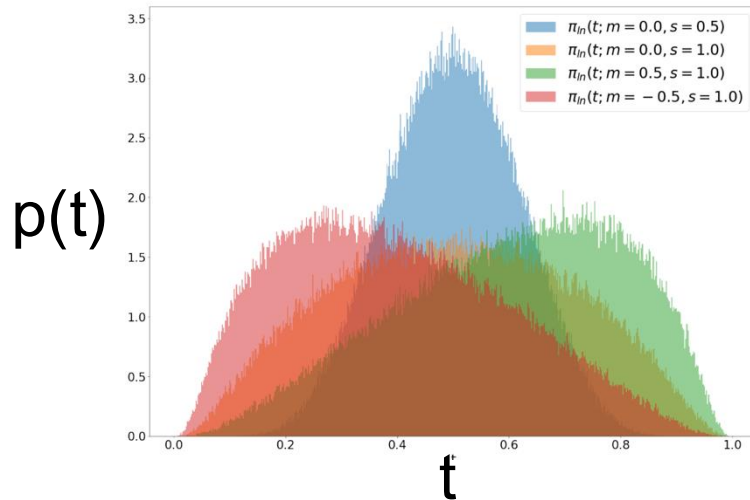
Full noise ( $t=1$ ) is easy: optimal  $v$  is mean of  $p_{\text{data}}$

No noise ( $t=0$ ) is easy: optimal  $v$  is mean of  $p_{\text{noise}}$

Middle noise is hardest, most ambiguous

But we give equal weight to all noise levels!

**Solution:** Use a non-uniform noise schedule



Put more emphasis on middle noise

# Noise Schedules

## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = torch.randn().sigmoid()
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

There may be many pairs  $(x, z)$  that give the same  $x_t$ ; network must average over them

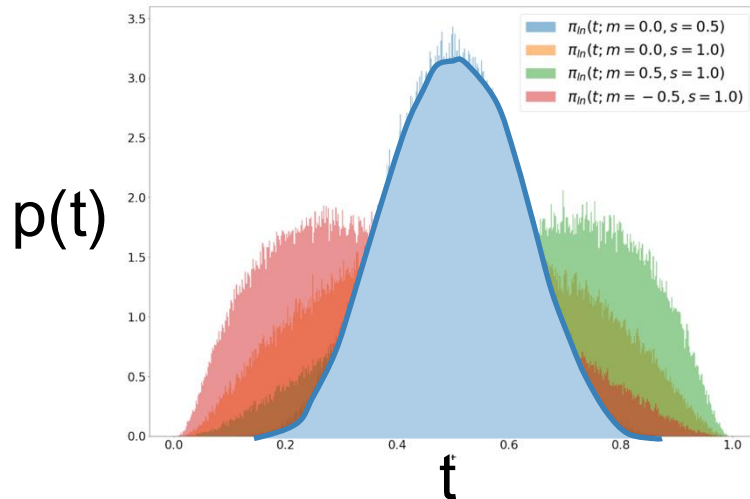
Full noise ( $t=1$ ) is easy: optimal  $v$  is mean of  $p_{\text{data}}$

No noise ( $t=0$ ) is easy: optimal  $v$  is mean of  $p_{\text{noise}}$

Middle noise is hardest, most ambiguous

But we give equal weight to all noise levels!

**Solution:** Use a non-uniform noise schedule



Put more emphasis on middle noise

Common choice: **logit-normal sampling**

# Noise Schedules

## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = torch.randn().sigmoid()
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

There may be many pairs  $(x, z)$  that give the same  $x_t$ ; network must average over them

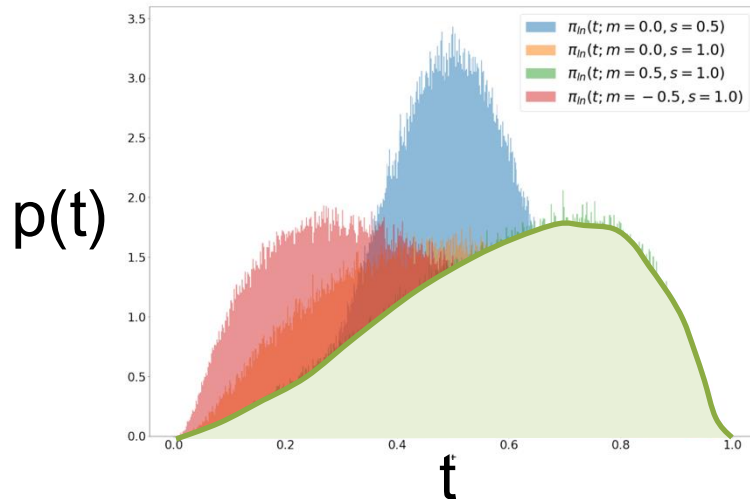
Full noise ( $t=1$ ) is easy: optimal  $v$  is mean of  $p_{\text{data}}$

No noise ( $t=0$ ) is easy: optimal  $v$  is mean of  $p_{\text{noise}}$

Middle noise is hardest, most ambiguous

But we give equal weight to all noise levels!

**Solution:** Use a non-uniform noise schedule



Put more emphasis on middle noise

Common choice: **logit-normal sampling**

For high-res data, often **shift to higher noise** to account for pixel correlations

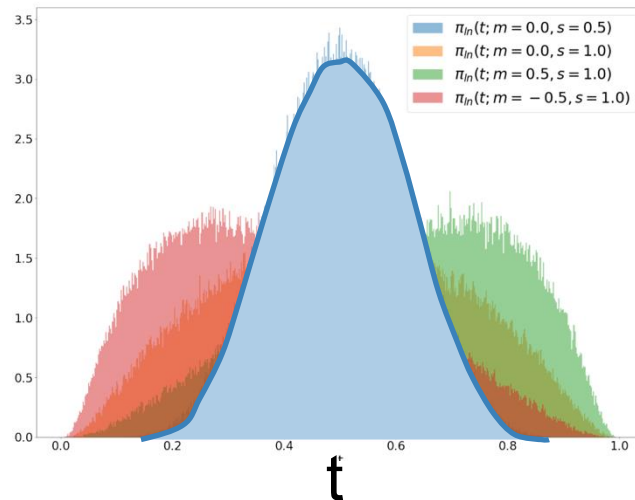
# Diffusion: Rectified Flow

## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = torch.randn().sigmoid()
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Simple and scalable setup  
for many generative  
modeling problems!

$p(t)$



## Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    vy = model(sample, y, t)
    v0 = model(sample, y_null, t)
    v = (1 + w) * vy - w * v0
    sample = sample - v / num_steps
```

Put more emphasis on middle noise

Common choice: **logit-normal sampling**

For high-res data, often **shift to higher noise** to account for pixel correlations

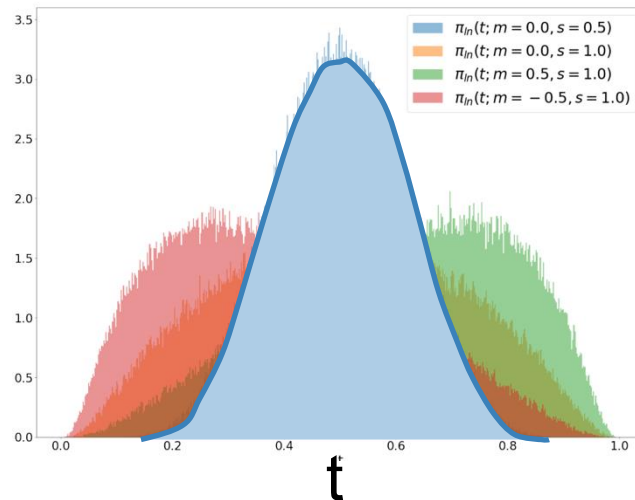
# Diffusion: Rectified Flow

## Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = torch.randn().sigmoid()
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Simple and scalable setup  
for many generative  
modeling problems!

$p(t)$



## Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    vy = model(sample, y, t)
    v0 = model(sample, y_null, t)
    v = (1 + w) * vy - w * v0
    sample = sample - v / num_steps
```

**Problem:** Does not  
work naively on high-  
resolution data

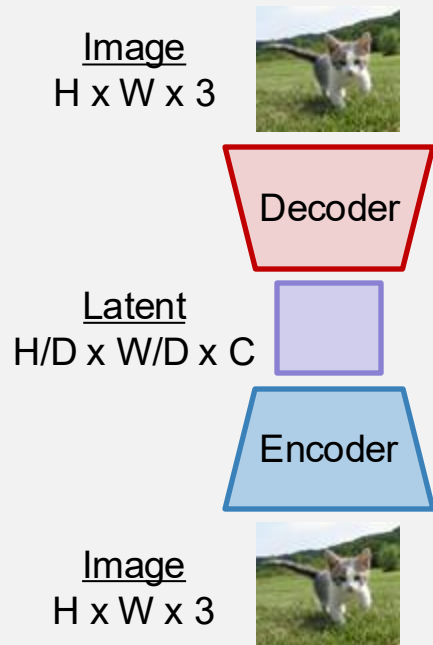
Put more emphasis on middle noise

Common choice: **logit-normal sampling**

For high-res data, often **shift to higher noise** to account for pixel correlations

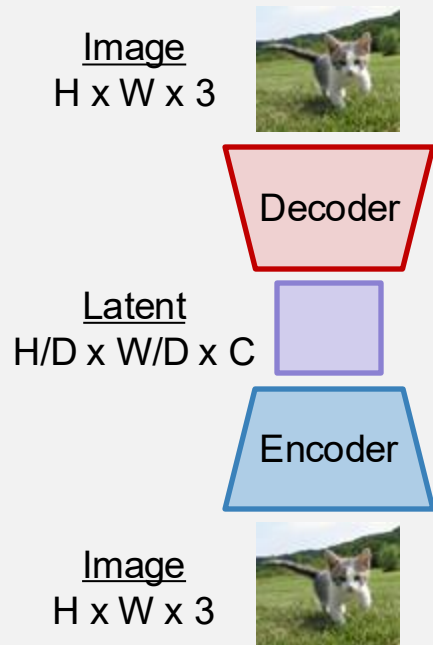
# Latent Diffusion Models (LDMs)

Train **encoder** + **decoder** to  
convert images to **latents**



# Latent Diffusion Models (LDMs)

Train **encoder** + **decoder** to  
convert images to **latents**



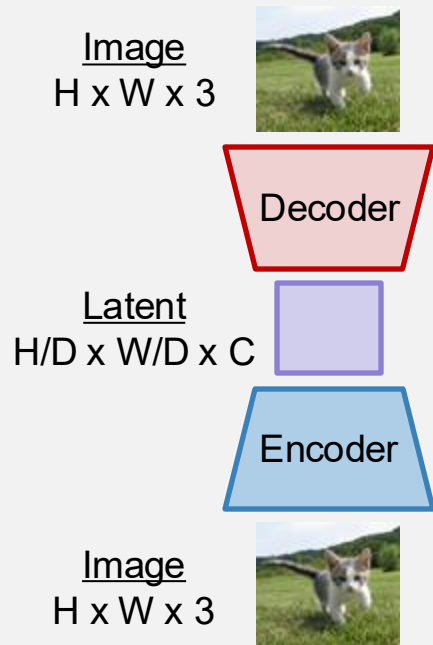
Common setting:  $D=8$ ,  $C=16$

**Image:**  $256 \times 256 \times 3$   
 $\Rightarrow$  **Latent:**  $32 \times 32 \times 16$

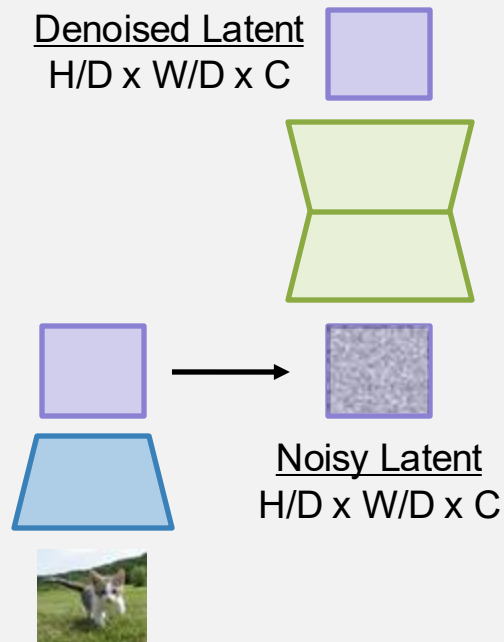
Encoder / Decoder are CNNs with attention

# Latent Diffusion Models (LDMs)

Train **encoder** + **decoder** to  
convert images to **latents**

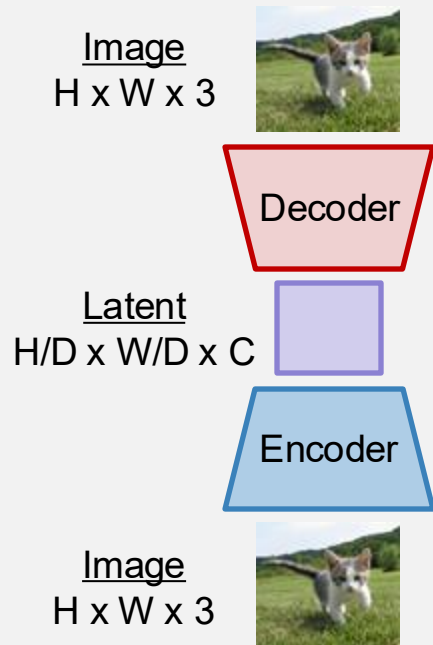


Train **diffusion model** to  
remove noise from **latents**  
(**Encoder** is frozen)

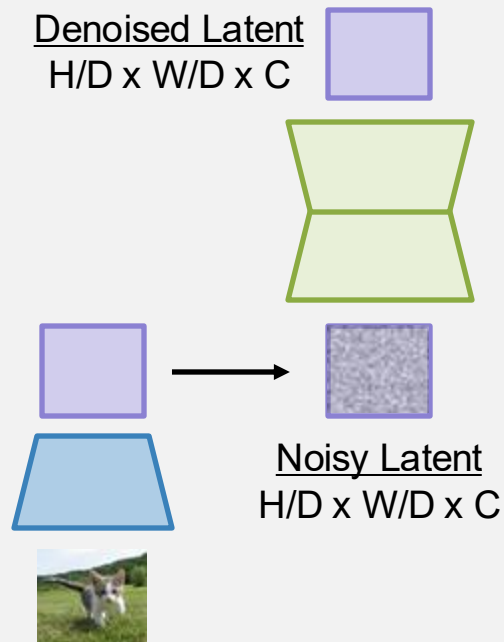


# Latent Diffusion Models (LDMs)

Train **encoder** + **decoder** to  
convert images to **latents**



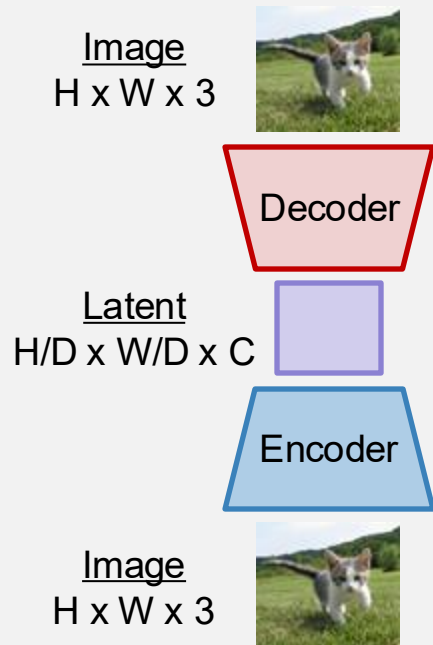
Train **diffusion model** to  
remove noise from **latents**  
(**Encoder** is frozen)



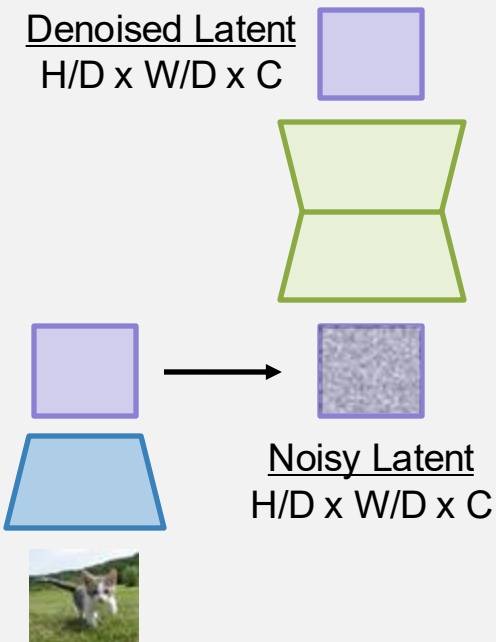
After training:

# Latent Diffusion Models (LDMs)

Train **encoder** + **decoder** to  
convert images to **latents**



Train **diffusion model** to  
remove noise from **latents**  
(**Encoder** is frozen)



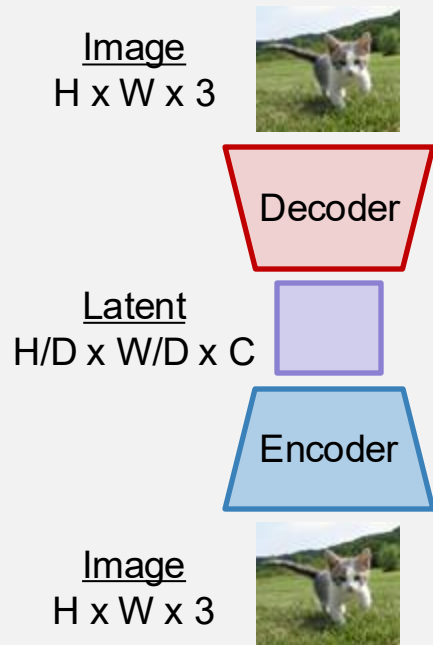
After training:

Sample random  
**latent**

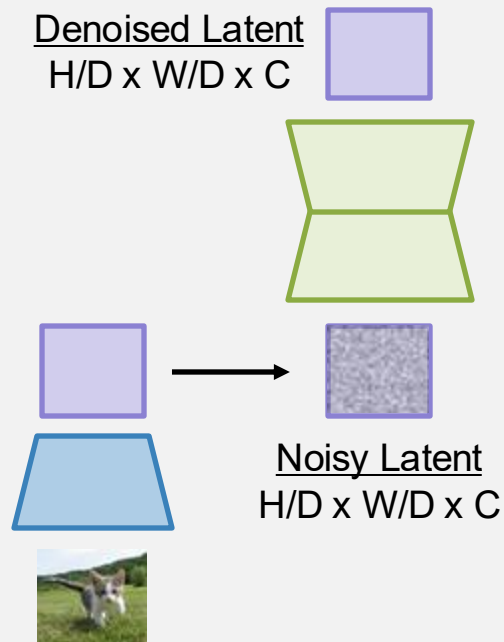


# Latent Diffusion Models (LDMs)

Train **encoder** + **decoder** to  
convert images to **latents**



Train **diffusion model** to  
remove noise from **latents**  
(**Encoder** is frozen)



After training:

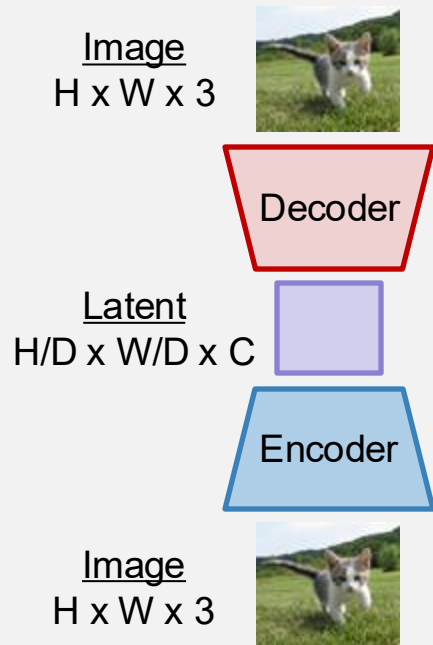
Sample random  
**latent**

Iteratively apply  
**diffusion model**  
to remove noise

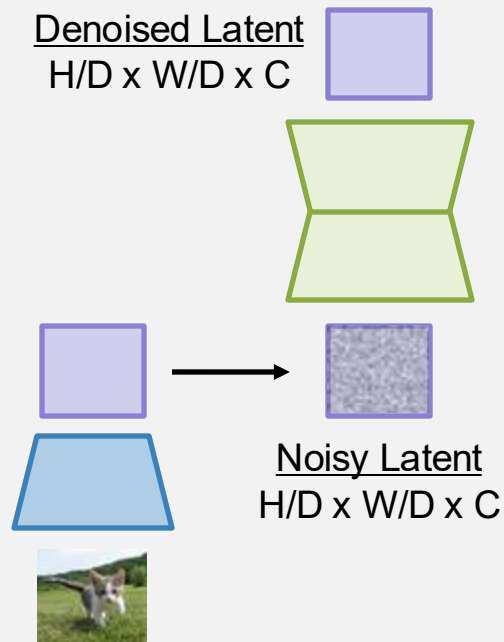


# Latent Diffusion Models (LDMs)

Train **encoder** + **decoder** to  
convert images to **latents**



Train **diffusion model** to  
remove noise from **latents**  
(**Encoder** is frozen)

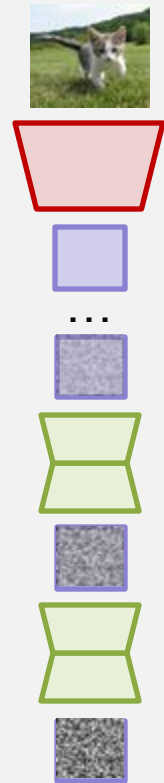


After training:

Sample random  
**latent**

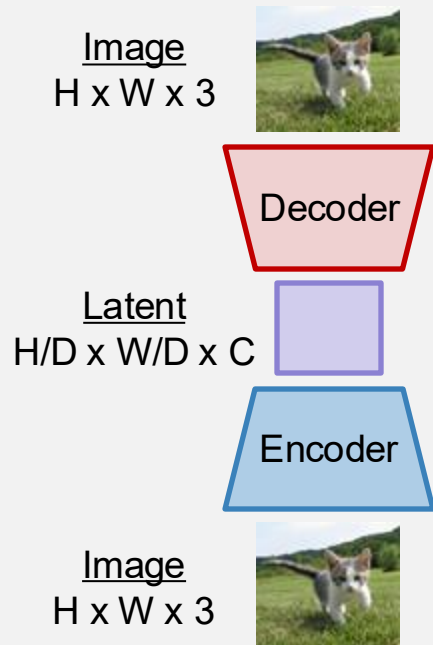
Iteratively apply  
**diffusion model**  
to remove noise

run **decoder** to  
get **image**

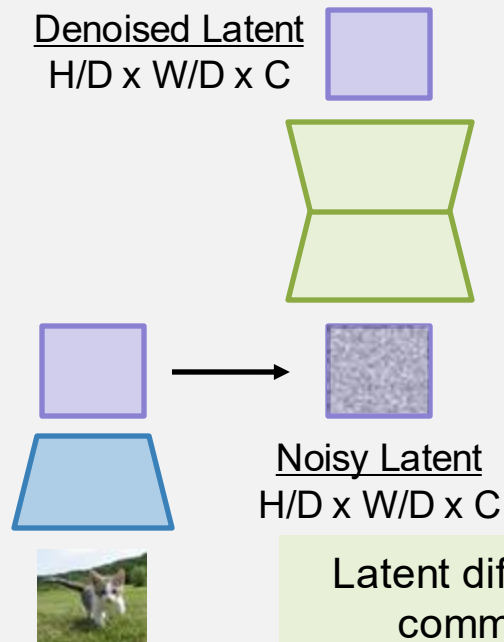


# Latent Diffusion Models (LDMs)

Train **encoder** + **decoder** to  
convert images to **latents**



Train **diffusion model** to  
remove noise from **latents**  
(**Encoder** is frozen)

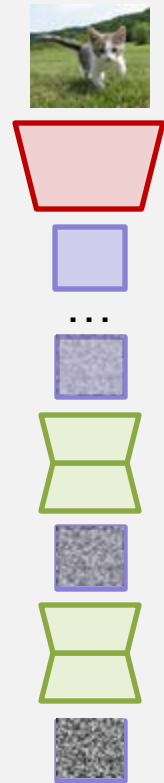


After training:

Sample random  
**latent**

Iteratively apply  
**diffusion model**  
to remove noise

run **decoder** to  
get **image**

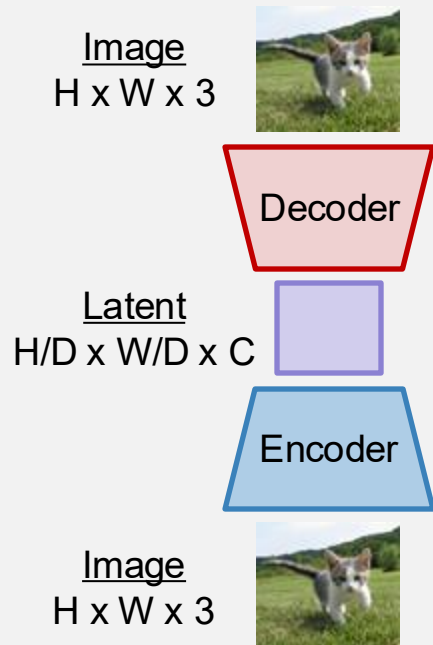


Latent diffusion is the most  
common form today

# Latent Diffusion Models (LDMs)

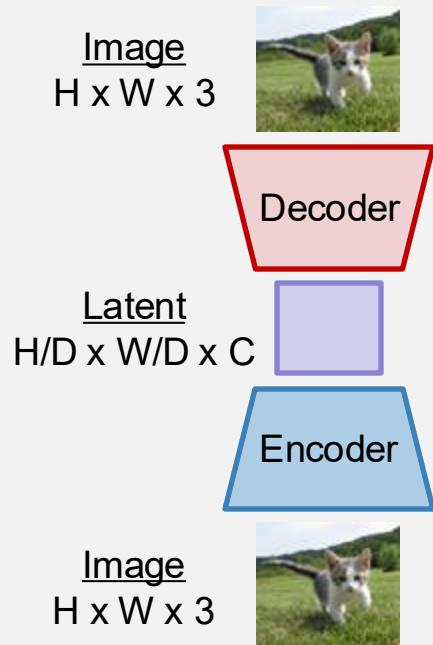
Train **encoder** + **decoder** to  
convert images to **latents**

How do we train the  
encoder+decoder?



# Latent Diffusion Models (LDMs)

Train **encoder** + **decoder** to  
convert images to **latents**

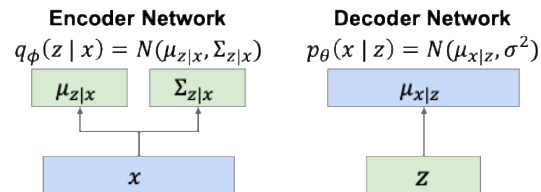


How do we train the  
encoder+decoder?

**Solution: It's a VAE!**  
Typically with very  
small KL prior weight

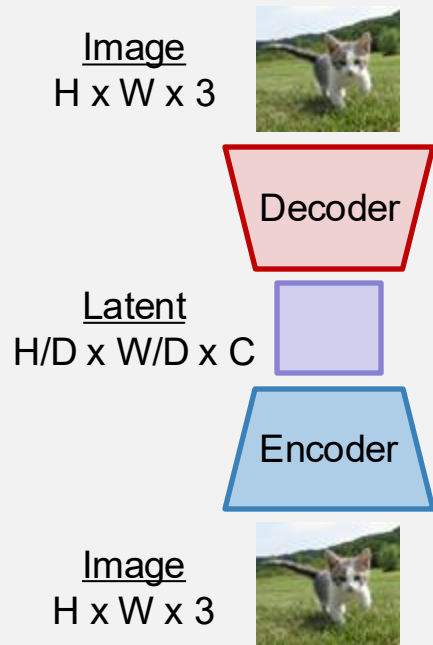
Recall: VAE

$$\log p_{\theta}(x) \geq E_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x), p(z))$$



# Latent Diffusion Models (LDMs)

Train **encoder** + **decoder** to  
convert images to **latents**



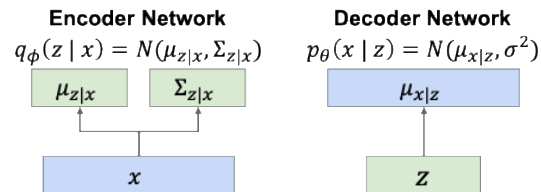
How do we train the  
encoder+decoder?

**Solution:** It's a VAE!  
Typically with very  
small KL prior weight

**Problem:** Decoder  
outputs often blurry

Recall: VAE

$$\log p_{\theta}(x) \geq E_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x), p(z))$$



# Latent Diffusion Models (LDMs)

How do we train the  
encoder+decoder?

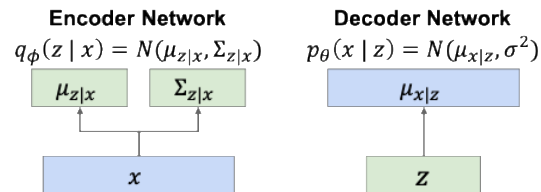
**Solution: It's a VAE!**  
Typically with very  
small KL prior weight

**Problem: Decoder  
outputs often blurry**

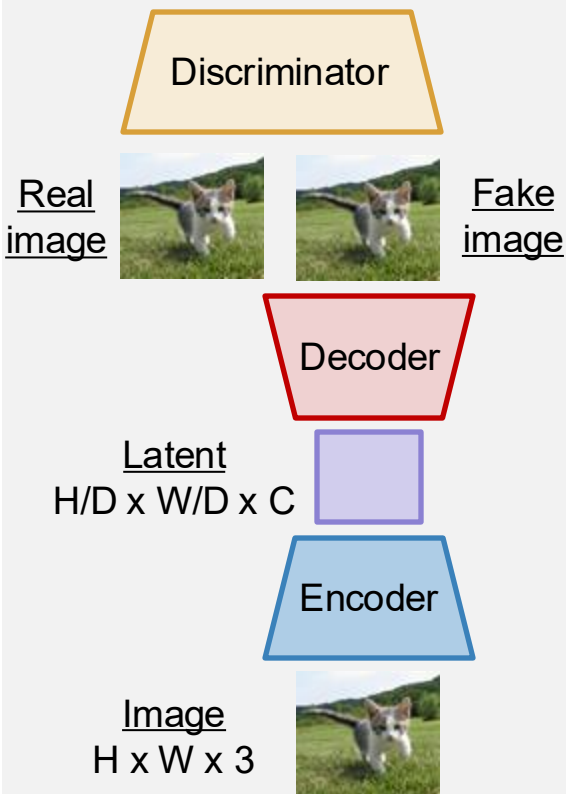
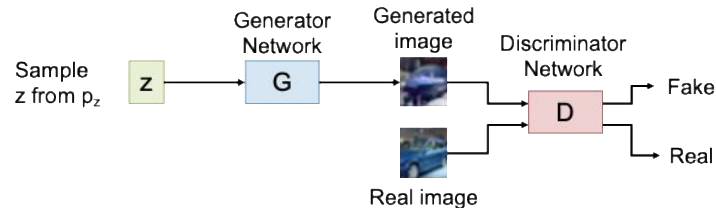
**Solution: Add a  
discriminator!**

Recall: VAE

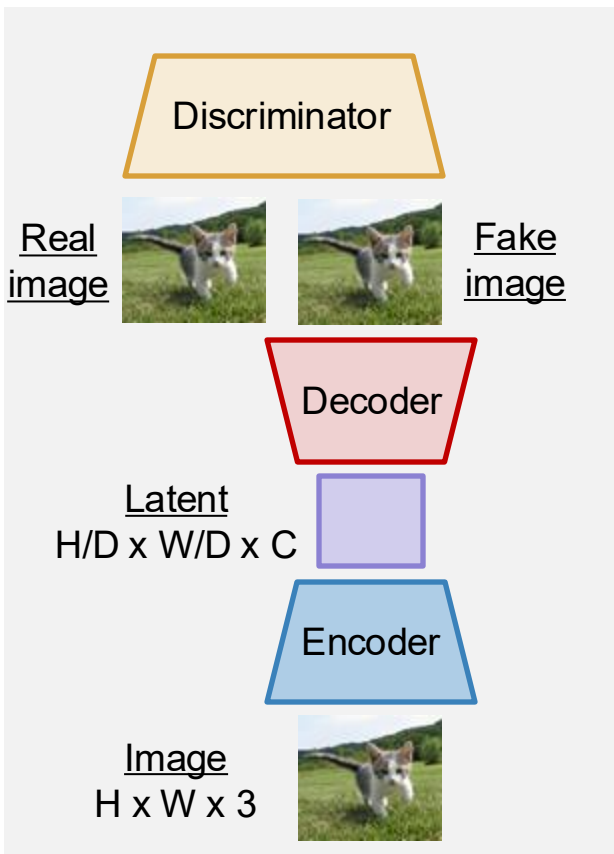
$$\log p_{\theta}(x) \geq E_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x), p(z))$$



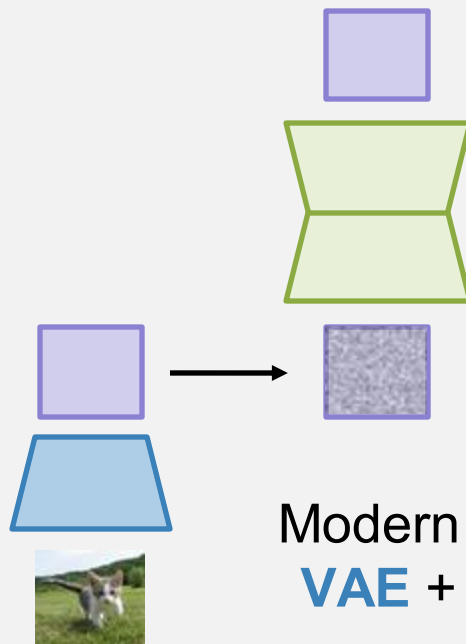
Recall: GAN



# Latent Diffusion Models (LDMs)



Train **diffusion model** to  
remove noise from **latents**  
(**Encoder** is frozen)



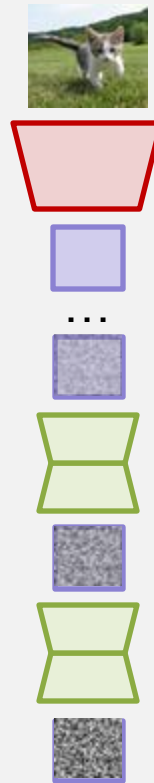
Modern LDM pipelines use  
**VAE** + **GAN** + **diffusion**!

After training:

Sample random  
**latent**

Iteratively apply  
**diffusion model**  
to remove noise

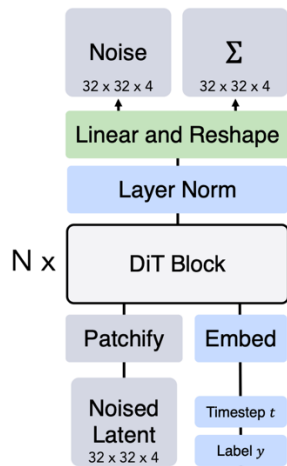
run **decoder** to  
get **image**



# Diffusion Transformer (DiT)

Diffusion uses standard Transformer blocks!

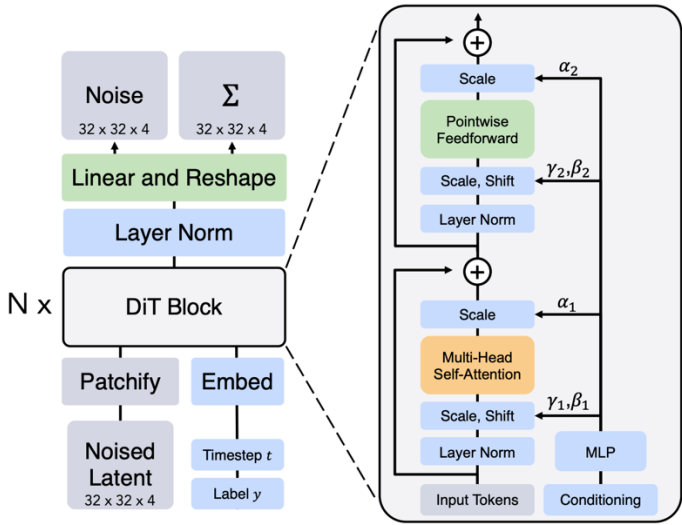
Main question: How to inject conditioning (timestep  $t$ , text, ...)



# Diffusion Transformer (DiT)

Diffusion uses standard Transformer blocks!

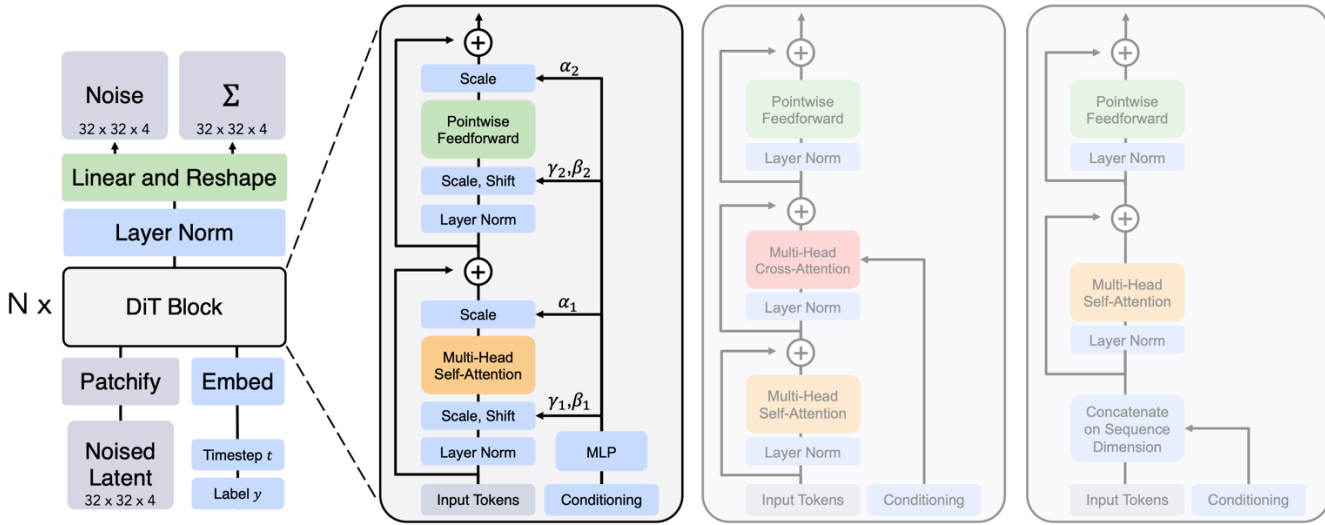
Main question: How to inject conditioning (timestep  $t$ , text, ...)



**Predict scale/shift:**  
Most common for  
diffusion timestep  $t$

# Diffusion Transformer (DiT)

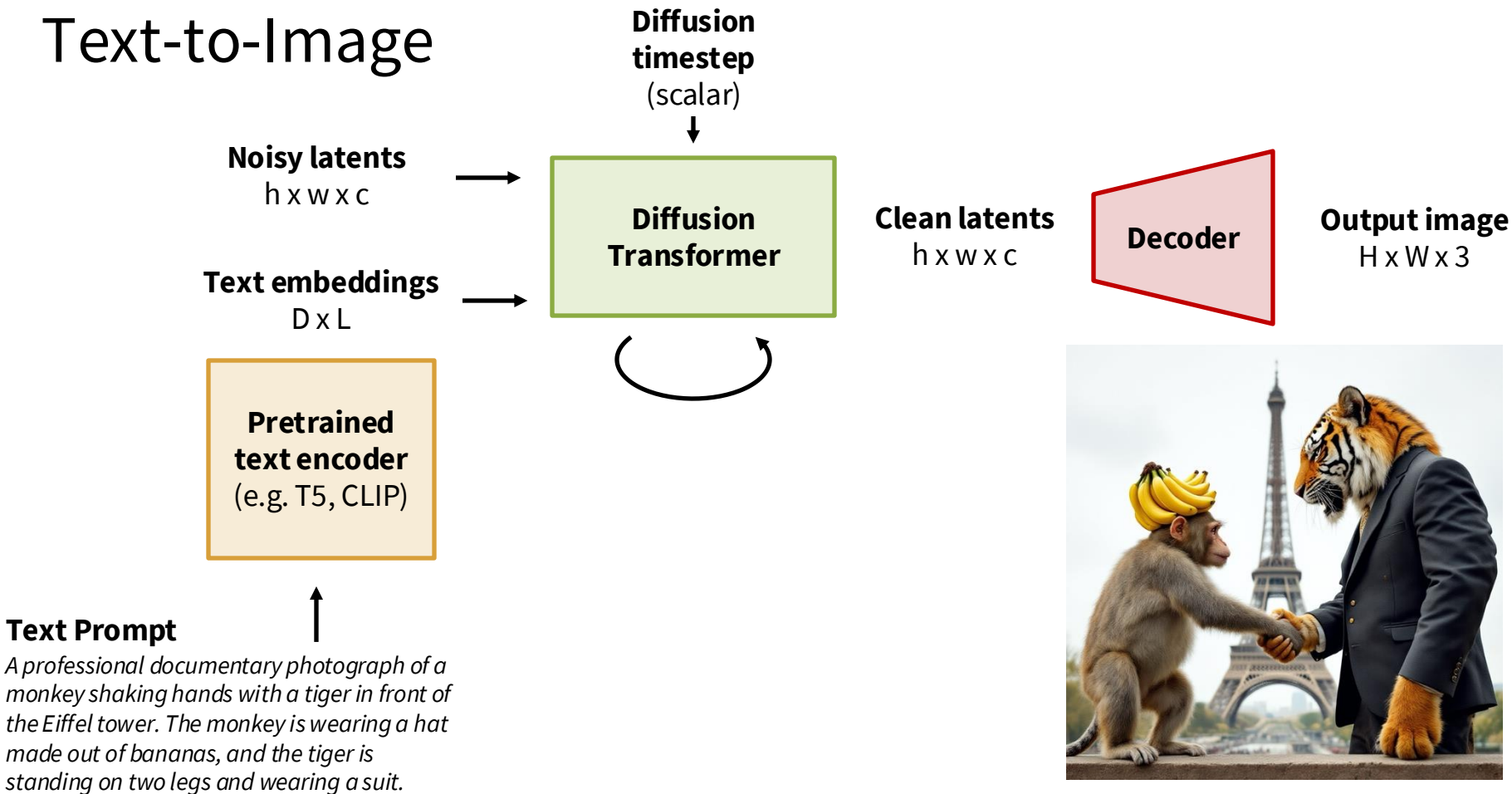
Diffusion uses standard Transformer blocks!  
Main question: How to inject conditioning (timestep  $t$ , text, ...)



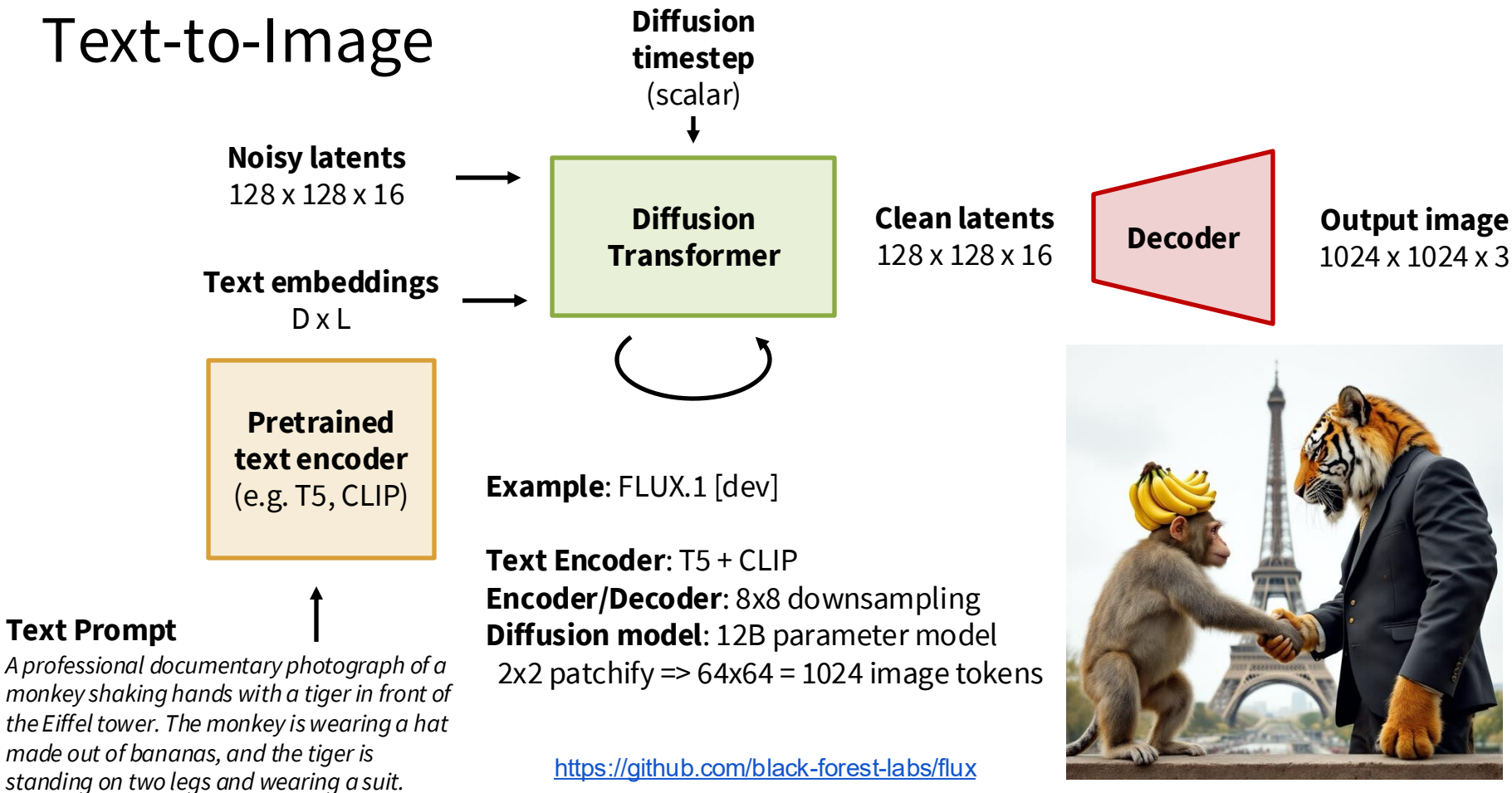
**Predict scale/shift:**  
Most common for  
diffusion timestep  $t$

**Cross-Attention / Joint Attention:**  
Common for text, image, etc conditioning

# Text-to-Image

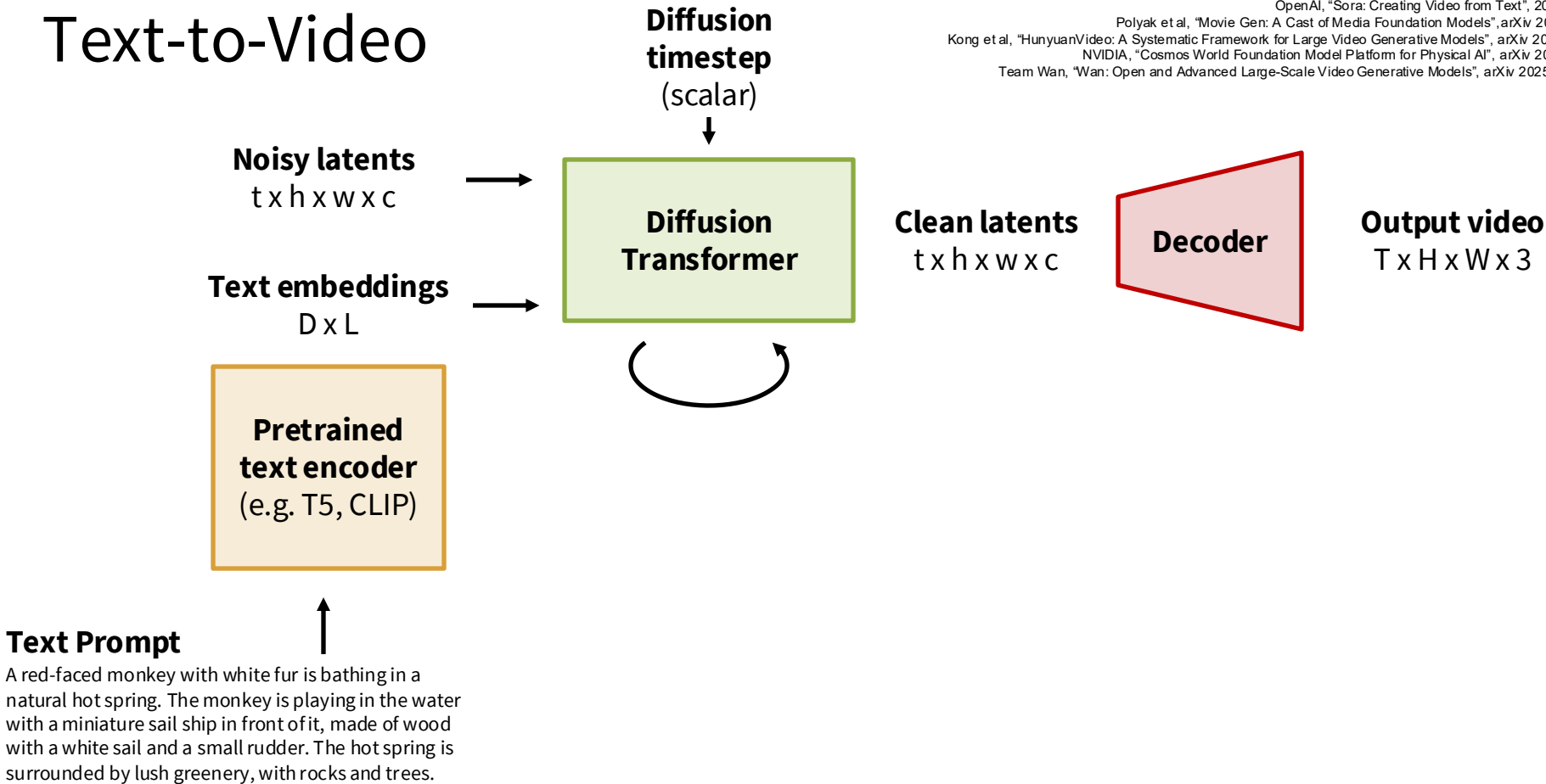


# Text-to-Image



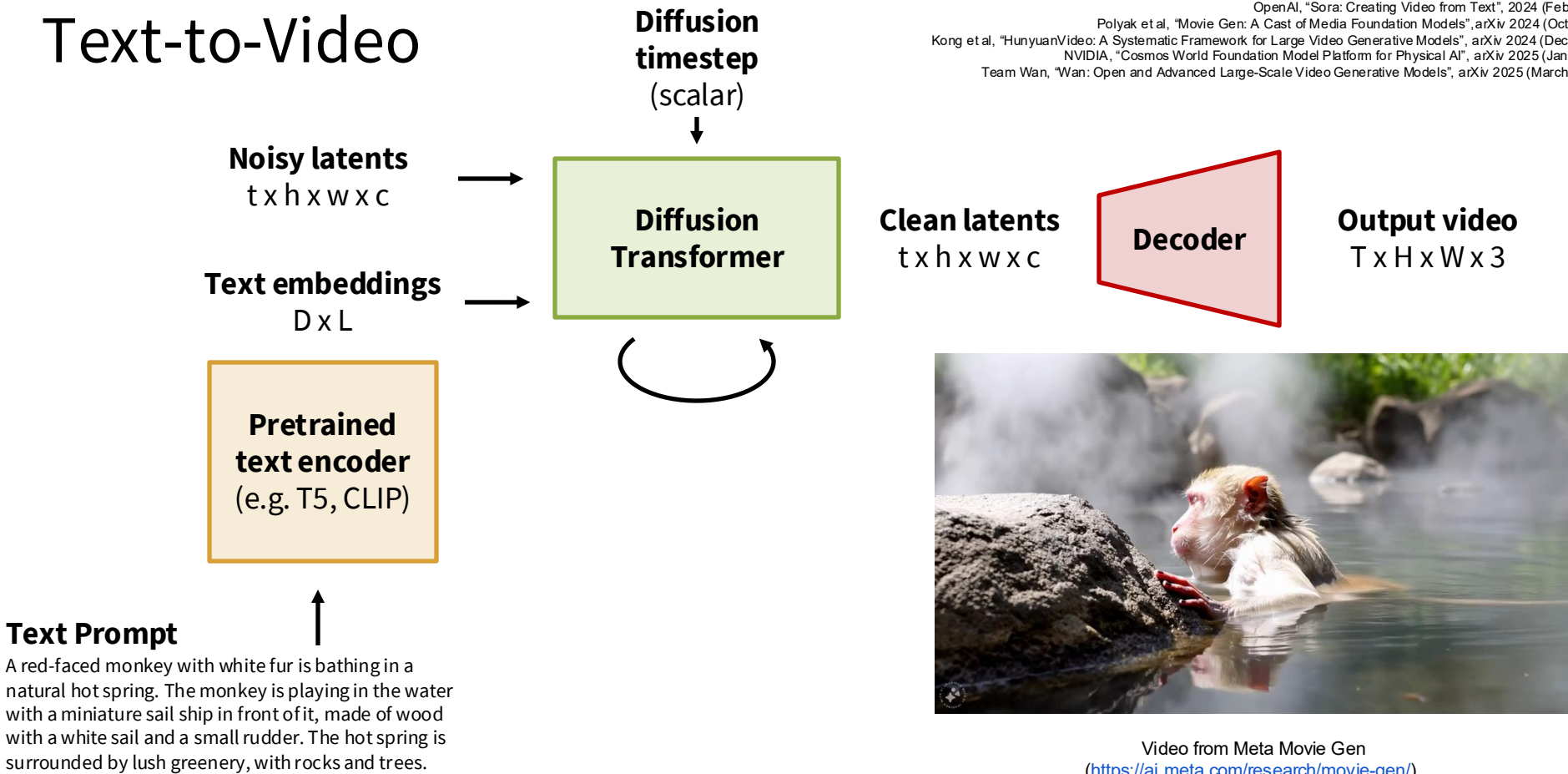
# Text-to-Video

Gupta et al, "Photorealistic Video Generation with Diffusion Models", arXiv 2023 (Dec)  
OpenAI, "Sora: Creating Video from Text", 2024 (Feb)  
Polyak et al, "Movie Gen: A Cast of Media Foundation Models", arXiv 2024 (Oct)  
Kong et al, "HunyuanVideo: A Systematic Framework for Large Video Generative Models", arXiv 2024 (Dec)  
NVIDIA, "Cosmos World Foundation Model Platform for Physical AI", arXiv 2025 (Jan)  
Team Wan, "Wan: Open and Advanced Large-Scale Video Generative Models", arXiv 2025 (March)



# Text-to-Video

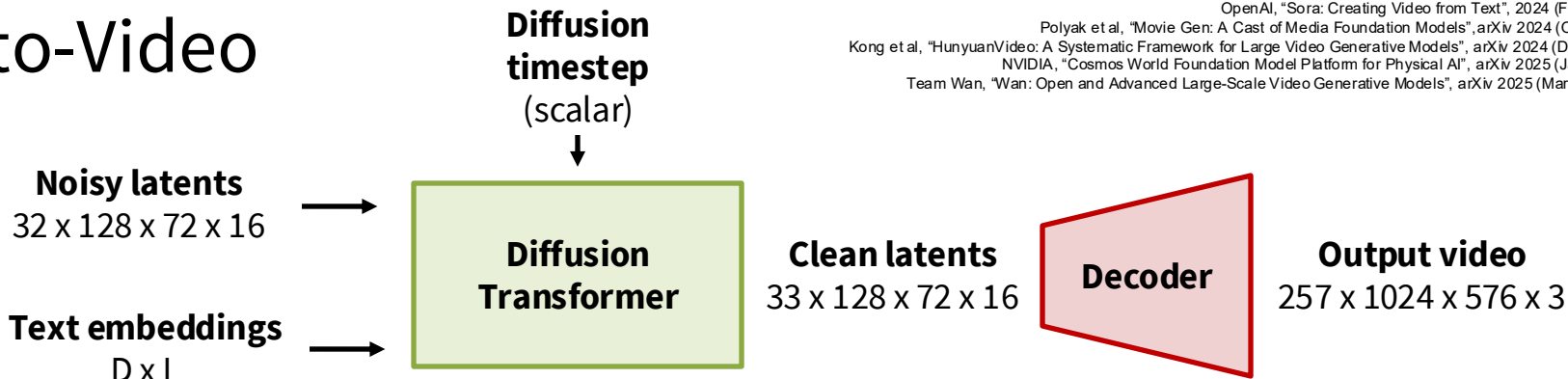
Gupta et al, "Photorealistic Video Generation with Diffusion Models", arXiv 2023 (Dec)  
OpenAI, "Sora: Creating Video from Text", 2024 (Feb)  
Polyak et al, "Movie Gen: A Cast of Media Foundation Models", arXiv 2024 (Oct)  
Kong et al, "HunyuanVideo: A Systematic Framework for Large Video Generative Models", arXiv 2024 (Dec)  
NVIDIA, "Cosmos World Foundation Model Platform for Physical AI", arXiv 2025 (Jan)  
Team Wan, "Wan: Open and Advanced Large-Scale Video Generative Models", arXiv 2025 (March)



Video from Meta Movie Gen  
(<https://ai.meta.com/research/movie-gen/>)

# Text-to-Video

Gupta et al, "Photorealistic Video Generation with Diffusion Models", arXiv 2023 (Dec)  
OpenAI, "Sora: Creating Video from Text", 2024 (Feb)  
Polyak et al, "Movie Gen: A Cast of Media Foundation Models", arXiv 2024 (Oct)  
Kong et al, "HunyuanVideo: A Systematic Framework for Large Video Generative Models", arXiv 2024 (Dec)  
NVIDIA, "Cosmos World Foundation Model Platform for Physical AI", arXiv 2025 (Jan)  
Team Wan, "Wan: Open and Advanced Large-Scale Video Generative Models", arXiv 2025 (March)



**Example:** Meta MovieGen

**Text Encoder:** UL2, ByT5, MetaCLIP

**Encoder/Decoder:** 8x8x8 downsample

**Diffusion model:** 30B param DiT

1x2x2 patchify => 76K tokens



## Text Prompt

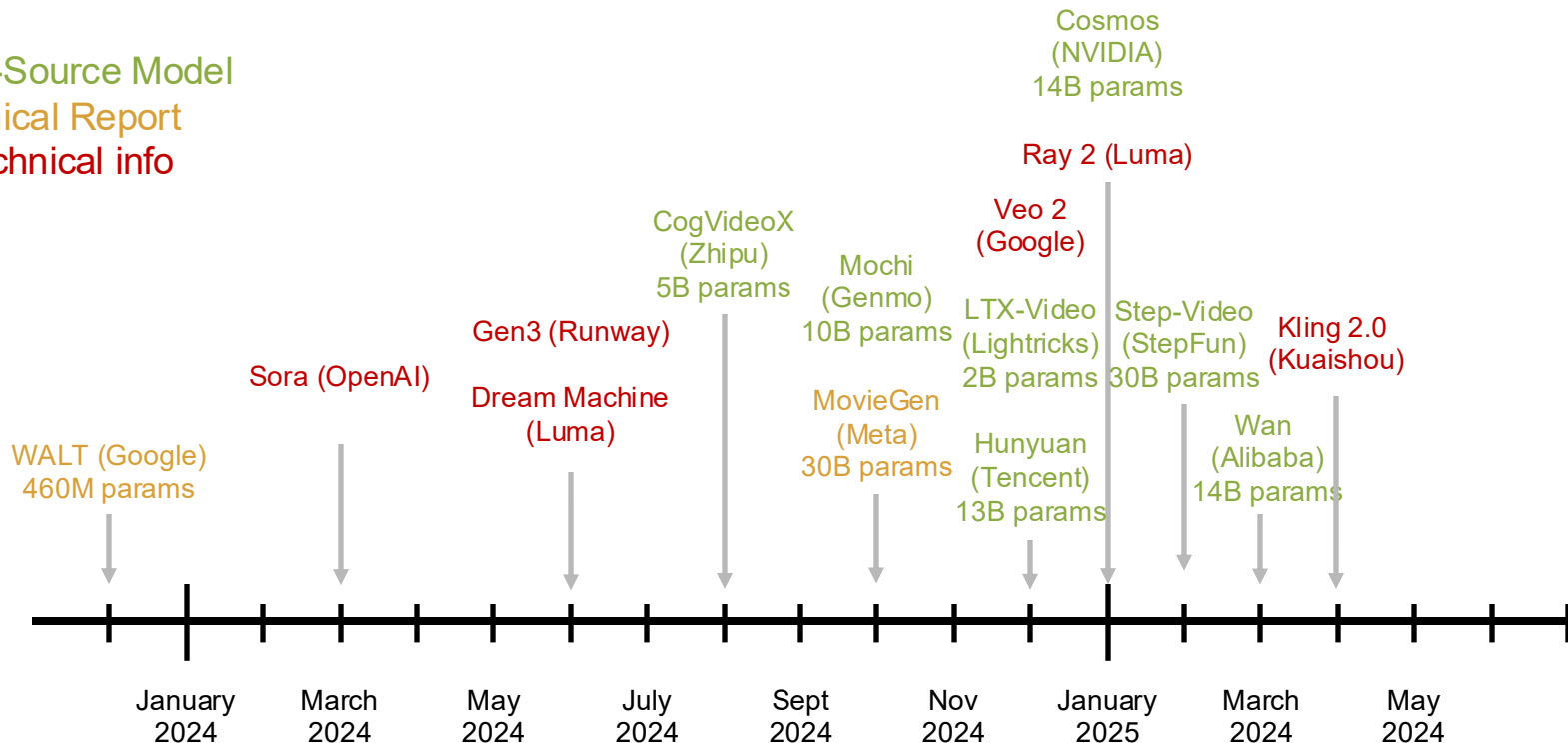
A red-faced monkey with white fur is bathing in a natural hot spring. The monkey is playing in the water with a miniature sail ship in front of it, made of wood with a white sail and a small rudder. The hot spring is surrounded by lush greenery, with rocks and trees.

# The Era of Video Diffusion Models

Open-Source Model

Technical Report

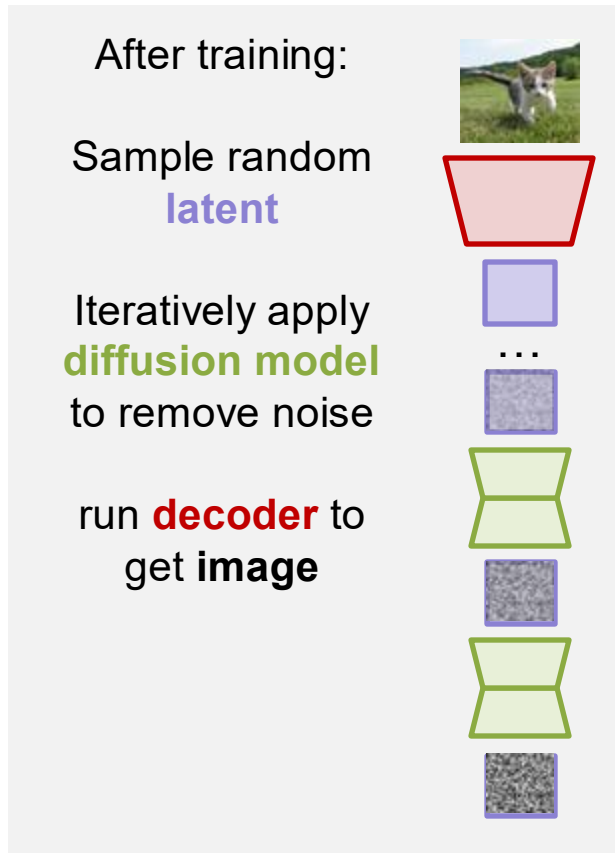
No technical info



# Diffusion Distillation

During sampling we need to run the diffusion model many times (~30 – 50 for rectified flow)

This is really slow!



# Diffusion Distillation

During sampling we need to run the diffusion model many times (~30 – 50 for rectified flow)

This is really slow!

**Solution:** **distillation** algorithms reduce the number of steps (sometimes all the way to 1), can also bake in CFG

After training:

Sample random  
**latent**

Iteratively apply  
**diffusion model**  
to remove noise

run **decoder** to  
get **image**



...



Salimans and Ho, "Progressive Distillation for Fast Sampling of Diffusion Models", ICLR 2022  
Song et al, "Consistency Models", ICML 2023  
Sauer et al, "Adversarial Diffusion Distillation", ECCV 2024  
Sauer et al, "Fast High-Resolution Image Synthesis with Latent Adversarial Diffusion Distillation", arXiv 2024  
Lu and Song, "Simplifying, Stabilizing and Scaling Consistency Models", ICLR 2025  
Salimans et al, "Multistep Distillation of Diffusion Models via Moment Matching", NeurIPS 2025

# Generalized Diffusion

## Rectified Flow

Sample  $x \sim p_{data}$ ,  $z \sim p_{noise}$

Sample  $t \sim p_t$

Set  $x_t = (1 - t)x + tz$

Set  $v_{gt} = z - x$

Compute  $v_{pred} = f_{\theta}(x_t, t)$

Compute loss  $\|v_{gt} - v_{pred}\|_2^2$

# Generalized Diffusion

## Rectified Flow

Sample  $x \sim p_{data}$ ,  $z \sim p_{noise}$

Sample  $t \sim p_t$

Set  $x_t = (1 - t)x + tz$

Set  $v_{gt} = z - x$

Compute  $v_{pred} = f_{\theta}(x_t, t)$

Compute loss  $\|v_{gt} - v_{pred}\|_2^2$



## Generalized Diffusion

Sample  $x \sim p_{data}$ ,  $z \sim p_{noise}$

Sample  $t \sim p_t$

Set  $x_t = a(t)x + b(t)z$

Set  $y_{gt} = c(t)x + d(t)z$

Compute  $y_{pred} = f_{\theta}(x_t, t)$

Compute loss  $\|y_{gt} - y_{pred}\|_2^2$

# Generalized Diffusion

## Rectified Flow

Sample  $x \sim p_{data}$ ,  $z \sim p_{noise}$

Sample  $t \sim p_t$

Set  $x_t = (1 - t)x + tz$

Set  $v_{gt} = z - x$

Compute  $v_{pred} = f_{\theta}(x_t, t)$

Compute loss  $\|v_{gt} - v_{pred}\|_2^2$

$$a(t) = 1 - t$$

$$b(t) = t$$

$$c(t) = -1$$

$$d(t) = 1$$



## Generalized Diffusion

Sample  $x \sim p_{data}$ ,  $z \sim p_{noise}$

Sample  $t \sim p_t$

Set  $x_t = a(t)x + b(t)z$

Set  $y_{gt} = c(t)x + d(t)z$

Compute  $y_{pred} = f_{\theta}(x_t, t)$

Compute loss  $\|y_{gt} - y_{pred}\|_2^2$

# Generalized Diffusion

## Variance Preserving (VP)

$$a(t) = \sqrt{\sigma(t)}$$

$$b(t) = \sqrt{1 - \sigma(t)}$$

If  $x$  and  $z$  are independent and variance=1, then  $x_t$  also has variance=1

## Generalized Diffusion

Sample  $x \sim p_{data}$ ,  $z \sim p_{noise}$

Sample  $t \sim p_t$

Set  $x_t = a(t)x + b(t)z$

Set  $y_{gt} = c(t)x + d(t)z$

Compute  $y_{pred} = f_{\theta}(x_t, t)$

Compute loss  $\|y_{gt} - y_{pred}\|_2^2$

# Generalized Diffusion

## Variance Exploding (VE)

$$a(t) = 1$$

$$b(t) = \sigma(t)$$

$\sigma(1)$  Needs to be big enough  
to drown out all signal in  $x$

## Generalized Diffusion

Sample  $x \sim p_{data}$ ,  $z \sim p_{noise}$

Sample  $t \sim p_t$

Set  $x_t = a(t)x + b(t)z$

Set  $y_{gt} = c(t)x + d(t)z$

Compute  $y_{pred} = f_{\theta}(x_t, t)$

Compute loss  $\|y_{gt} - y_{pred}\|_2^2$

# Generalized Diffusion

x-prediction

$$y_{gt} = x \quad [\mathbf{c}(t) = 1; \mathbf{d}(t) = 0]$$

$\varepsilon$ -prediction

$$y_{gt} = z \quad [\mathbf{c}(t) = 0; \mathbf{d}(t) = 1]$$

v-prediction

$$y_{gt} = \mathbf{b}(t)z - \mathbf{a}(t)x \quad [\mathbf{c}(t) = \mathbf{b}(t); \mathbf{d}(t) = -\mathbf{a}(t)]$$

## Generalized Diffusion

Sample  $x \sim p_{data}$ ,  $z \sim p_{noise}$

Sample  $t \sim p_t$

Set  $x_t = \mathbf{a}(t)x + \mathbf{b}(t)z$

Set  $y_{gt} = \mathbf{c}(t)x + \mathbf{d}(t)z$

Compute  $y_{pred} = f_{\theta}(x_t, t)$

Compute loss  $\|y_{gt} - y_{pred}\|_2^2$

# Generalized Diffusion

How do we choose these functions?

Usually through some **mathematical formalism**

## Generalized Diffusion

Sample  $x \sim p_{data}$ ,  $z \sim p_{noise}$

Sample  $t \sim p_t$

Set  $x_t = a(t)x + b(t)z$

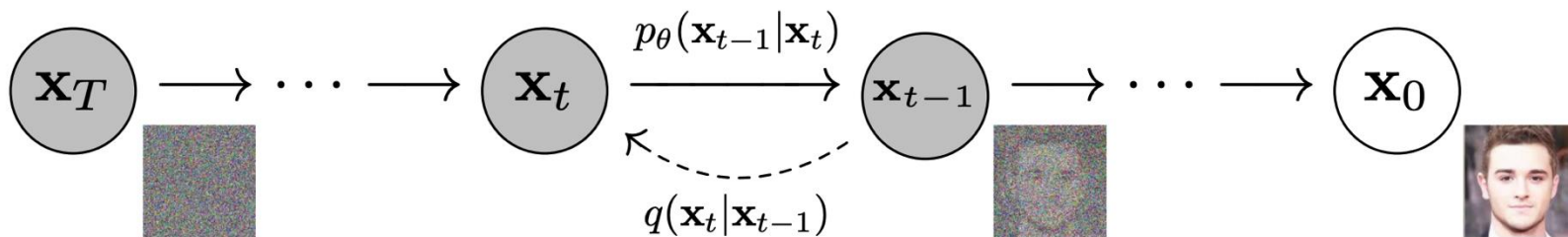
Set  $y_{gt} = c(t)x + d(t)z$

Compute  $y_{pred} = f_{\theta}(x_t, t)$

Compute loss  $\|y_{gt} - y_{pred}\|_2^2$

# Diffusion is a Latent Variable Model

We know the forward process: Add Gaussian noise



Learn a network to approximate the backward process

Optimize variational lower bound (same as VAE)

# Diffusion Learns the Score Function

For any distribution  $p(x)$  over  $x \in \mathbb{R}^N$  the **score function**

$$s: \mathbb{R}^N \rightarrow \mathbb{R}^N \quad s(x) = \frac{\partial}{\partial x} \log p(x)$$

Is a vector field pointing toward areas of high probability density

Diffusion learns a neural network to approximate the score function of  $p_{\text{data}}$

# Diffusion Solves Stochastic Differential Equations

We can describe a continuous noising process as an SDE

$$d\mathbf{x} = f(\mathbf{x}, t)dt + g(t)d\mathbf{w}$$

Gives a relationship between infinitesimal changes in data  $\mathbf{x}$ , time  $t$ , and noise  $\mathbf{w}$ .

Diffusion learns a neural network to approximately solve this SDE

# Perspectives on Diffusion

1. *Diffusion models are autoencoders*
2. *Diffusion models are deep latent variable models*
3. *Diffusion models predict the score function*
4. *Diffusion models solve reverse SDEs*
5. *Diffusion models are flow-based models*
6. *Diffusion models are recurrent neural networks*
7. *Diffusion models are autoregressive models*
8. *Diffusion models estimate expectations*

Great blog post by Sander Dieleman:

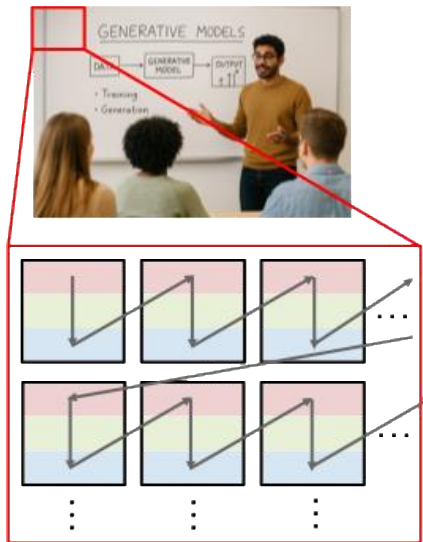
<https://sander.ai/2023/07/20/perspectives.html>

(All his blog posts are great)

# Autoregressive Models Strike Back

Recall autoregressive models

Too slow on raw pixels



They work great on  
(discrete) latents!

# Autoregressive Models Strike Back

Train **encoder** + **decoder**  
to convert images to  
discrete latents

Image  
 $H \times W \times 3$



Decoder

Latent  
 $H/D \times W/D$   
integers!

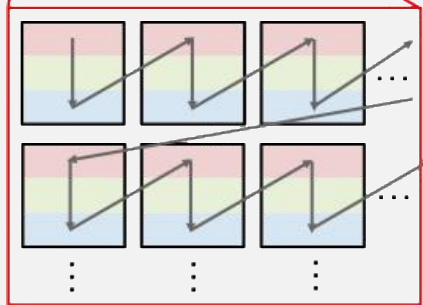


Encoder

Image  
 $H \times W \times 3$



Train **autoregressive model**  
to model sequences of  
discrete latents

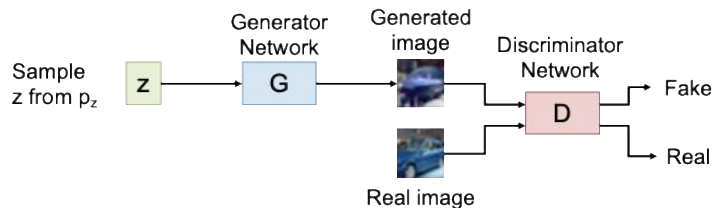


Sample discrete latents from  
the **autoregressive model**,  
pass to **decoder** to get an  
image

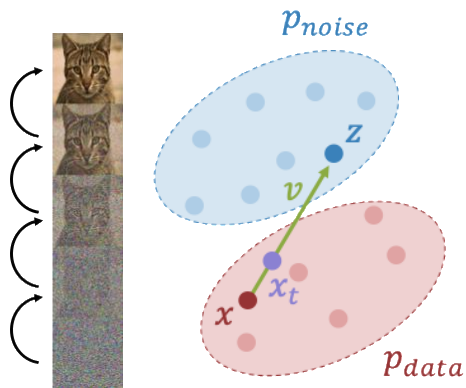
van den Oord et al, "Neural Discrete Representation Learning", NeurIPS 2017  
Razavi et al, "Generating Diverse High-Fidelity Images with VQ-VAE-2", NeurIPS 2019  
Esser et al, "Taming Transformers for High-Resolution Image Synthesis", CVPR 2021  
Yu et al, "Scaling Autoregressive Models for Content-Rich Text-to-Image Generation", arXiv 2022

# Summary

## Generative Adversarial Networks



## Diffusion Models



## Latent Diffusion Models

